Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Робототехника и комплексная автоматизация»

КАФЕДРА «Компьютерные системы автоматизации производства»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ

НА ТЕМУ:

«Система планирования и диспетчеризации линии окраски пластиковых деталей»

Студент РК9-81Б _____ (Подпись, дата) Д.И. Максимов

Руководитель ВКР _____ (Подпись, дата) М.В. Овсянников

Консультант _____ (Подпись, дата) С.А. Буханов

Нормоконтролер _____ (Подпись, дата) Г.В. Малинин

2025 г.

РЕФЕРАТ

Выпускная квалификационная работа выполнена по теме: «Система планирования и диспетчеризации линии окраски пластиковых деталей».

Расчетно-пояснительная записка содержит 51 лист формата А4, 10 рисунков, 3 таблицы, 16 используемых источников, 2 приложения.

Целью выполнения работы является разработка и реализация алгоритма оптимизации планирования покраски пластиковых деталей с учетом технологических ограничений и требований к производственному процессу. В работе проведен анализ существующей системы, выявлены ее недостатки, проанализированы современные методы планирования и диспетчеризации производственных процессов и основных алгоритмов теории расписания, разработано алгоритмическое решение задачи оптимизации планирования, реализован программный код для автоматизации процесса планирования расписания покраски и проведено тестирование разработанной системы на производственных данных.

Объектом исследования является линия окраски пластиковых деталей автомобильного завода АО МАЗ «Москвич».

В работе применялось следующее программное обеспечение: текстовый редактор Microsoft Word, программа для работы с электронными таблицами Microsoft Excel, сервис построения блок-схем draw.io, редактор кода Microsoft Visual Studio Code.

Ключевые слова: линия окраски пластиковых деталей, планирование и диспетчеризация, расписание, Python, теория расписаний, алгоритм оптимизации.

СОДЕРЖАНИЕ

РЕФЕРАТ 2

ВВЕДЕНИЕ 5

1. Предпроектное обследование 6

1.1. Целевое обследование линии окраски 6

1.1.1. Краткое техническое описание процесса покраски пластиковых деталей 7

1.2. Описание и анализ системы планирования и диспетчеризации линии окраски пластиковых деталей 13

2. Концептуальное проектирование 18

2.1. Цели и задачи разработки 18

2.2. Анализ существующих решений в области планирования и диспетчеризации производств 20

2.3. Теория расписаний. Выбор методов и алгоритмов для планирования расписания линии окраски 24

2.3.1 Математическая формализация модели 26

2.3.2 Применимые методы и алгоритмы для решения задачи 26

2.4. Выбор языка программирования 29

3. Техническое задание 33

3.1. Общие сведения 33

3.2. Назначение и цели создания системы 33

3.3. Требования к системе в целом 34

4. Структурное проектирование 35

4.1. Разработка блок-схемы основных алгоритмов работы программы 35

5. Рабочее проектирование 38

5.1. Разработка программного кода 38
6. Апробирование 44
6.1. Тестирование работы программы 44
ЗАКЛЮЧЕНИЕ 48
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 50
ПРИЛОЖЕНИЕ А 52
ПРИЛОЖЕНИЕ Б 63

ВВЕДЕНИЕ

Современные тенденции развития промышленности диктуют необходимость повышения эффективности производственных процессов за счет внедрения передовых технологий управления и автоматизации. В условиях жесткой конкуренции на мировом рынке и роста требований к качеству и надежности автомобилей особое внимание уделяется совершенствованию технологических процессов в автомобилестроении, обеспечивающих высокое качество конечной продукции.

Окраска является одним из ключевых этапов производства, от которого зависит внешний вид и долговечность автомобиля. Эффективное планирование и диспетчеризация линий окраски позволяет существенно повысить конкурентоспособность завода и качество выпускаемых автомобилей, что особенно важно для таких предприятий, как АО МАЗ «Москвич» .

Актуальность данной темы обусловлена необходимостью разработки и внедрения современных методов оптимизации систем планирования и диспетчеризации линий окраски пластиковых деталей с учетом приоритетов сборки, минимизации переналадок и своевременно обеспечения комплектности деталей для сборки кузовов. Эффективная организация линии способствует рациональному использованию производственных ресурсов, улучшению качества изделий и снижению себестоимости конечного продукта, что, в свою очередь, укрепляет конкурентные позиции завода «Москвич» на рынке и способствует устойчивому развитию отечественной автомобильной промышленности.

1. Предпроектное обследование

1.1. Целевое обследование линии окраски

В рамках исследовательской работы производилось обследование объекта автоматизации в цеху по окраске пластиковых деталей на производстве АО МАЗ “Москвич”. Согласно технической документации [1] линия окраски пластиковых деталей представляет из себя цепной конвейер длиной 179.2 метра, на который установлены 64 тележки для заготовок. На каждой из телег располагается специальный суппорт для закрепления заготовок. Линия окрашивает 3 типа деталей для одной и той же модели автомобиля: бампера, транклиды и спойлеры (для каждого типа детали свой суппорт). На данный момент существую 5 цветов для окраса. В данный момент порядок назначенных тележек постоянен (суппорта не переставляют).

На данный момент конвейер двигается со средней скоростью 1.78 метров/минуту, что составляет 37-38 телег в час. Схема линии покраски представлена на рисунке 1.

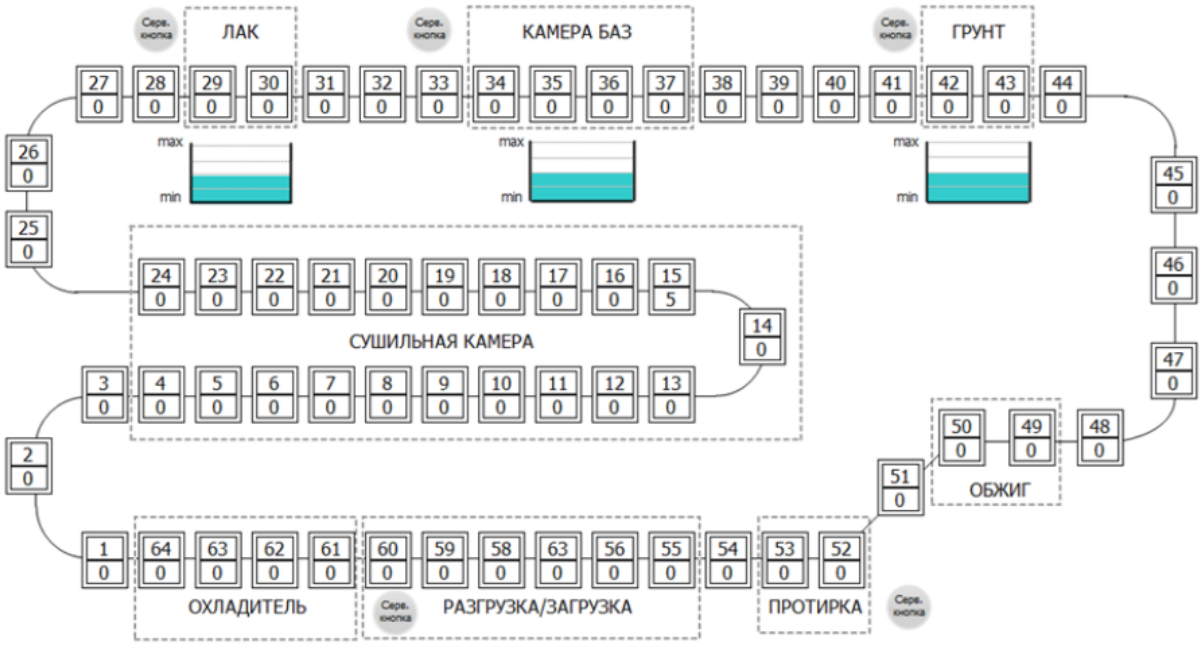


Рисунок 1 – Схема линии окраски пластиковых деталей АО МАЗ “Москвич”

1.1.1. Краткое техническое описание процесса покраски пластиковых деталей

1. Зона загрузки

В зоне загрузки осуществляется распаковка и установка заготовок на суппорты, предназначенные для движения по производственной линии. Установка одной заготовки совершается одним оператором (всего на линии 4 оператора). Общий вид поста отражен на рисунке 2:

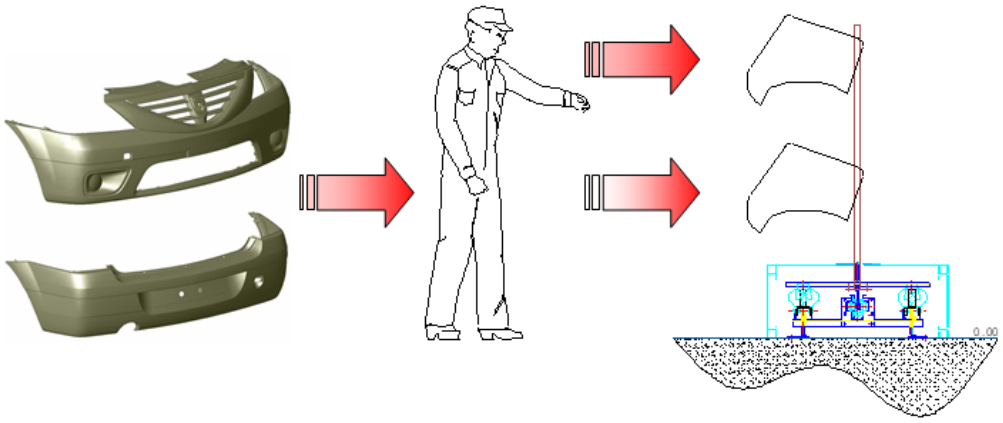


Рисунок 2 – Зона загрузки

2. Зона протирки

Зона протирки включает в себя продув и протирку бамперов. Продувка предназначена для очищения бамперов от пыли при помощи ионизирующего воздушного пистолета. Протирка бамперов от пыли осуществляется при помощи специального материала.

Один оператор обдувает и протирает заготовки, установленные на тележке (стальная рама с креплениями для пластиковых деталей, двигающаяся по линии окраски). Эта зона находится внутри кабины из оцинкованной стали, оборудованной приточно-вытяжной вентиляцией. Начиная с этого поста (зоны, доступной операторам линии), бампера входят в закрытую зону, защищенную от внешней пыли. Общий вид зоны протирки:

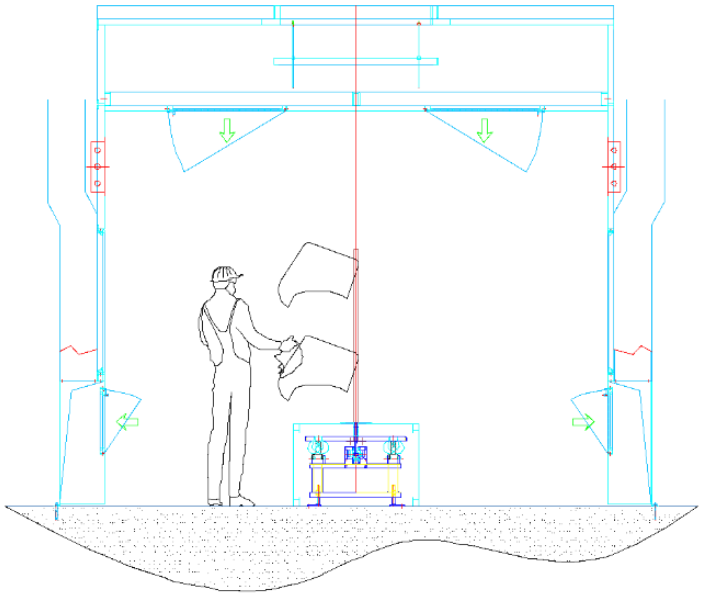


Рисунок 3 – Зона протирки

3. Кабина связи протирка/обжиг

Кабина связи протирки и обжига соединяет кабину протирки и пост обжига, заготовки перемещаются по конвейеру. Кабина защищена от попадания внешней пыли и грязи. Операторов в этой зоне нет. Эта зона выполнена из оцинкованной стали и соединена с вентиляцией.

4. Роботизированный пост обжига

В зоне обжига осуществляется обжиг бамперов газовой горелкой при помощи робота фирмы ABB. Обжиг поверхности бамперов осуществляется во время движения конвейера (без остановки) на расстоянии 10 – 15 см. от заготовок. Остальные типы деталей не обжигаются, робот находится в режиме ожидания. Процесс обжига происходит без присутствия операторов. Зона

защищена от доступа людей защитными ограждениями и системой безопасности.

Рисунок 4 – Робот фирмы ABB, оснащенный газовой горелкой

5. Кабина связи обжиг/покрытия

Кабина связи обжига и нанесения покрытия обеспечивает соединение кабины обжига и кабины нанесения покрытия, заготовки перемещаются на суппортах по конвейеру. Зона защищена от попадания внешней пыли и грязи. Операторов в этой зоне нет. Эта зона выполнена из оцинкованной стали и соединена с вентиляцией, пол кабины выполнен из металлических решеток, которые обеспечивают вытяжку.

6. Камеры нанесения покрытия

В камерах нанесения покрытия расположены три камеры для нанесений различных покрытий на заготовки – грунта, базы и лака. Принципы нанесения каждого слоя схожи. Покрытие наносят роботы фирмы Sames на расстоянии 5 – 10 см от заготовок без остановки конвейера. Во время нанесения слоев в камере создаётся сильнейший приток и отток воздуха (расчетно около 50 000 $м^3$/час), а также для сбора излишней распыленной краски вдоль одной из стен

организована водяная завеса, не позволяющая частицам покрытия оседать на стенах камеры.

Вся зона выполнена из оцинкованной стали, соединена с вентиляцией и защищена от попадания внешней пыли и грязи. Операторы могут отслеживать показатели датчиков и устранять ошибки через внешний пульт управления, установленный рядом с камерой нанесения покрытия.

Рисунок 5 – Фото процесса нанесения покрытия роботом Sames

7. Кабина связи покрытия / сушильная печь

Зона связи покрытия и сушильной печи обеспечивает соединение камеры нанесения покрытия и сушильной печи, бампера перемещаются на суппортах по конвейеру. Зона защищена от попадания внешней пыли. Операции выполняются без присутствия оператора в камере. Эта зона выполнена из оцинкованной стали и соединена с вентиляцией. Пол кабины выполнен из металлических решеток, которые обеспечивают вытяжку.

В этой зоне происходит испарение растворителя с поверхности окрашенных бамперов, требуемое время испарения растворителя слоя лака - 5 минут.

8. Камера сушки покрытия

Сушильная печь обеспечивает полимеризирование краски при проходе в тепловом туннеле с температурой воздуха 80°С. Вход и выход бамперов из печи

осуществляется через воздушные завесы. В зоне сушке каждая заготовка проводит минимум 30 минут, что позволяет покрытию высохнуть без подтеков и пригаров.

Рисунок 6 – Фото камеры сушки (изнутри)

9. Охладитель и зона разгрузки

После прохождения зоны сушки, заготовки необходимо охладить. Для этого детали проходят небольшую камеру охлаждения, в которой обдуваются холодным потоком воздуха в течении нескольких минут.

В зоне разгрузки операторы контроля качества снимают детали с суппортов и тщательно оценивают качество покраски с использованием рабочих инструментов. При возникновении некритических дефектов рабочие удаляют их на месте, либо отправляют заготовку на ретушь и повторное перекрашивание. В случае сильных дефектов или нарушения целостности окрашенных деталей, их утилизируют. Общий вид поста разгрузки представлен на рисунке 7:

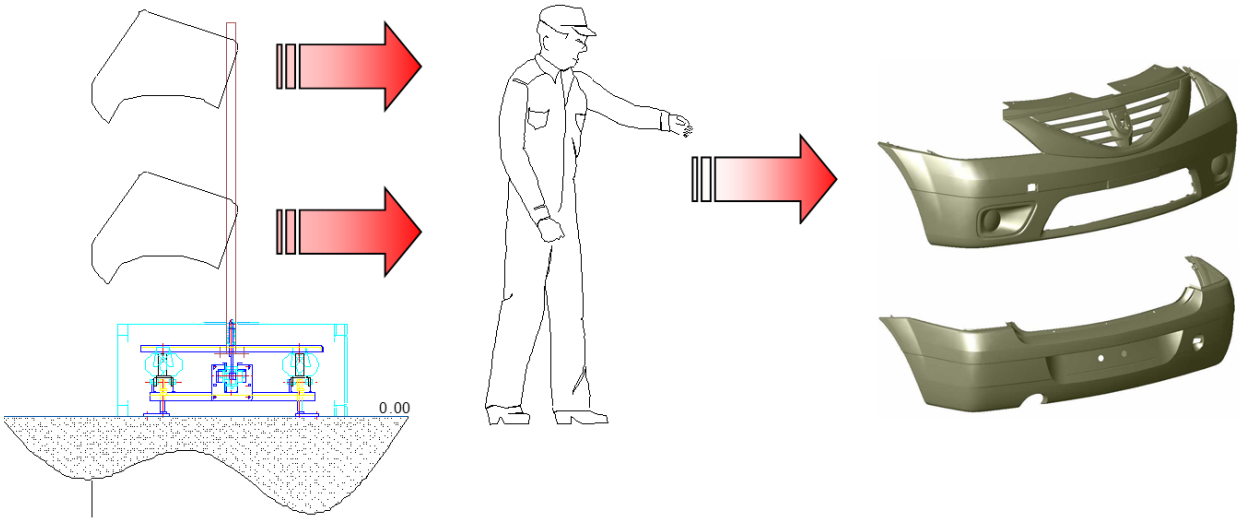


Рисунок 7 – Зона разгрузки

1.2. Описание и анализ системы планирования и диспетчеризации линии окраски пластиковых деталей

План на смену для линии окраски составляется ежедневно и зависит от данных (фильма), приходящих из цеха окраски кузовов. Фильм – таблица, содержащая в себе информацию о кузовах автомобилей, приходящих на сборку в течении смены, и обновляющаяся два раза в день: в 6:45 (начало смены) и в 13:00. Учет складских запасов ведется в программном обеспечении «1С» и обновляется автоматически в течении рабочего дня. По поступающей информации начальник участка и операторы на линии окраски определяют сколько комплектов деталей и какого цвета им нужно покрасить за смену. В конце рабочего дня собирается информация по фактическому производству и остаткам на складе, которые будут учтены на следующую смену.

На линии работают 4 оператора, которые решают в каком порядке по цветам будут окрашиваться детали и в какой момент производить смену цвета. Смена цвета и промывка производится автоматически роботом покраски, без остановки конвейера, но для нейтрализации опыла (микрочастиц краски, остающихся в воздухе после нанесения покрытия) операторы пропускают 1 –2 тележки между деталями разных цветов. В среднем за час по конвейеру проходит 37 – 38 тележек. Назначение тележек по виду загружаемой деталей описано на рисунке 8 и является постоянным (На одну тележку в зависимости от назначения устанавливается следующее количество деталей: бамперов по 2 штуки (1 комплект), спойлеров – по 2 штуки, транклидов – по 3 штуки). Заготовки окрашиваются партиями (по 13 тележек), смена цвета происходит только между партиями. В начале каждой смены начальник участка проверяет и подсчитывает количество тележек на линии и типы суппортов, установленных на них. Планирование на день ведется вручную, учет покрашенных и готовых деталей подсчитывается автоматически и поступает в систему, итоговый подсчет осуществляется в ручном режиме старшим оператором.

В зависимости от запаса окрашенных деталей на складе линия окраски может работать в двух режимах:

1) Если на складе имеется необходимый запас деталей нужных цветов на смену, то планирование и производство работают по системе “склад +1” (запасов покрашенных деталей на складе хватает на смену для отправки на сборку). В этом случае в цех сборки кузовов пластиковые детали поступают со склада, а на линии окраски производится заготовка деталей на следующие дни. При такой системе работы операторам удается минимизировать смену цветов и повысить эффективность работы.

2) Если на складе отсутствуют запасы, то производство идет в “онлайн” режиме: окрашенные комплекты деталей отправляются сразу на сборку. Чтобы удовлетворить запросы цеха кузовов и не допустить остановку сборки автомобилей, операторам линии окраски пластиковых деталей приходиться подстраиваться под поступающие требования и успевать отправить необходимый комплект деталей на сборку (от выхода кузова с монтажа до поступления его в цех сборки по лифту проходит примерно 2 часа, порядок поступления кузовов автомобилей по цветам указан в фильме). Эта система не предпочтительна, так как происходит большая потеря по проходам из-за частой смены цветов и пропуска тележек, тем самым понижается эффективность производства.

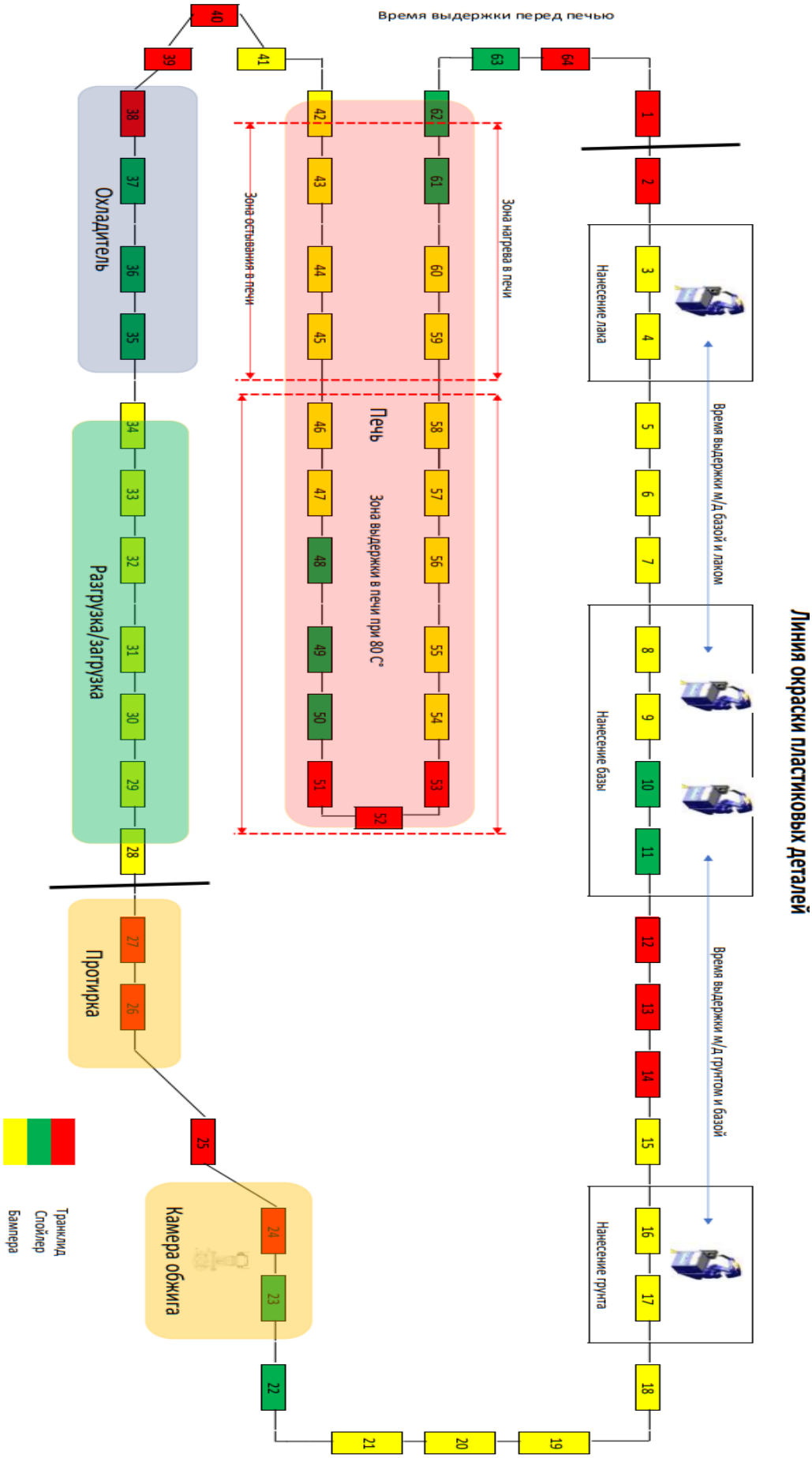


Рисунок 8 – Схема назначения тележек на линии окраски

При стандартном темпе работы всего завода за каждую смену необходимо произвести 126 комплектов пластиковых деталей различных цветов, что составляет примерно от 260 до 296 проходов тележек с учетом перекраса, брака, смены цветов, технических остановок работы роботов (примерно 20 – 30 минут в день) и двойного прохода и окраса транклид в силу производственных изъянов. Надежность линии с учетом остановок составляет 96%. С первого прохода на склад отправляют примерно 93 – 95% деталей, 5 – 7% составляют детали, которые отправляются на перекрас и ретушь, либо утилизируются как бракованные. Требуемые производственные мощности в зависимости от плана сборки автомобилей в смену представлены в таблице 1.

Таблица 1 – Требуемые производственные мощности

Требуемое количество телег с учетом двойного прохода транклид	**95 авто в смену**	**126 авто в смену**	**138 авто в смену**
Комплект бамперов на 1 авто (2 бампера)	95	126	138
Спойлер	47,5	63	69
Транклид	63,3	84	92
Итого	205,8	273,0	299,0
Процент перекраса	4%	4%	4%
Итого с учетом перекраса	214,0667	283,92	310,96
Требуемая производительность телег/час	33,8	44,9	49,1
Фактическая производительность телег/час	36,5	36,5	36,5
Коэффициент использования оборудования	93%	123%	135%
КИО без двойного прохода транклид	78%	104%	114%

С учетом нынешних производственных мощностей и планирования обеспечить полное выполнение стандартного плана смены в режиме “онлайн”

практически невозможно (по расчетам коэффициент использования оборудования без двойного прохода транклид составляет 104%, с двойным 123%).

Основными проблемами, возникающими на данный момент, являются потери по объему производства из-за отсутствия четкого планирования и частых переналадок (смен цвета), что несет за собой потерю по проходам тележек (особенно для работы в “онлайне”). Данные производственные проблемы не дают возможности стабильно удерживать и наполнять складской запас окрашенных деталей (система “склад +1”). Из этого складывается постоянная необходимость работы в сверхурочное время для выполнения плана, что является дополнительными финансовыми потерями и производственными нагрузками для автомобильного завода “Москвич”.

Решение данных производственных проблем позволит отделу линии окраски стабильно выполнять план на смену и удерживать складской запас без дополнительных переработок персонала.

2. Концептуальное проектирование

2.1. Цели и задачи разработки

Основная задача данной работы заключается в оптимизации процесса планирования и диспетчеризации линии окраски пластиковых деталей на автомобильном заводе «Москвич», что является важным этапом производственного цикла. В современных условиях автомобилестроения, характеризующихся высокой степенью автоматизации и жесткими требованиями к срокам и качеству, эффективное управление технологическими процессами становится ключевым фактором повышения производительности и снижения издержек

Целью данной работы является разработка алгоритма оптимизации порядка покраски партий пластиковых деталей с учетом технологических ограничений и требований к производственному процессу, а также создание программного обеспечения для реализации данного алгоритма.

Оптимизация процесса планирования и диспетчеризации поможет сократить нагрузку операторов на линии окраски пластиковых деталей и снизить вероятность ошибок из-за влияния человеческого фактора или изменения технологических параметров.

В ходе выполнения квалификационной работы были поставлены следующие **задачи**:

1. Анализ существующей системы планирования и диспетчеризации линии окраски пластиковых деталей;
2. Анализ современных методов планирования и диспетчеризации производственных процессов;
3. Анализ основных алгоритмов и методов теории расписания, применяющихся для планирования производств;
4. Разработка алгоритмического решения задачи оптимизации планирования на основе методов теории расписаний, способного учитывать сложные производственные ограничения и приоритеты;

5. Реализация программного кода для автоматизации процесса планирования расписания покраски;
6. Тестирование разработанной системы на реальных данных завода «Москвич» и оценка эффективности предложенного подхода.

2.2. Анализ существующих решений в области планирования и диспетчеризации производств

Планирование и диспетчеризация производственных линий – ключевые процессы управления современным промышленным производством, направленные на эффективное распределение ресурсов, минимизацию простоев оборудования и повышение производительности. В данной области существует несколько подходов и решений, которые уже применяются на практике:

1. Классическое MRP/ERP-планирование

Системы планирования потребностей в материалах (Material Requirements Planning (MRP)) и планирования ресурсов предприятия (Enterprise Resource Planning (ERP)) – это традиционные системы управления ресурсами предприятия, которые широко используются для планирования и диспетчеризации производственных процессов [2]. MRP фокусируется на управлении материалами и запасами, обеспечивая своевременное поступление необходимых компонентов для производства. Система рассчитывает потребности в материалах на основе плана производства, сроков поставки и текущих запасов.

ERP-системы расширяют функциональность MRP, интегрируя планирование производства с управлением финансами логистикой, персоналом и другими бизнес-процессами. Они обеспечивают централизованный контроль и координацию ресурсов, что способствует снижению издержек и повышению прозрачности производства.

Основными преимущества:

- ERP-система объединяет все процессы предприятия, что облегчает координацию и контроль;
- Все данные хранятся в одной системе, что упрощает доступ и анализ информации;
- Позволяет планировать производство на месяцы вперед, учитывая сезонные колебания спроса и другие внешние факторы.

Недостатки данных систем:

- Внедрение ERP-систем требует значительных временных и финансовых затрат. Настройка под конкретное предприятие может занять много времени;
- Классические ERP-системы могут быть недостаточно гибкими для оперативного управления изменениями, такими как внезапные сбои в производстве или изменения в заказах, что является критически важным требованием в нашей задаче.

2. Модели математического программирования

Для математического программирования используются методы линейного, нелинейного и целочисленного программирования для оптимизации расписаний работы производства. Данные методы позволяют формализовать задачу в виде оптимизационной модели с целью минимизации времени выполнения, затрат и других критериев. Линейное программирование используется для нахождения оптимального решения при наличии линейных ограничений и целевой функции. Например, можно рассчитать оптимальное распределение ресурсов для достижения максимальной прибыли. Целочисленное программирование применяется, когда переменные должны принимать целые значения, например, количество рабочих смен или единиц продукции. Нелинейное программирование используются для решения задач, где ограничения или целевая функция являются нелинейными. Это может включать, например, моделирование технологических процессов с учетом нелинейных зависимостей [3].

Основные преимущества:

- Математические модели позволяют учитывать множество факторов одновременно и находить оптимальные решения (даже в условиях неопределенности и вариабельности времени операций);
- Можно применять для самых разнообразных задач, от планирования загрузки оборудования до расчета оптимальных маршрутов транспортировки;

- Высокая вероятность нахождения оптимального решения.

Главные недостатки:

- Для сложных производственных процессов требуются значительные вычислительные мощности.
- Разработка и настройка математических моделей требует участия опытных квалифицированных специалистов.

3. Системы диспетчерского управления реального времени (RTDCS)

Системы диспетчерского управления реального времени (RTSCS) – это специализированные программные комплексы, которые позволяют управлять производственным процессом в реальном времени, оперативно реагируя на изменения в производственной среде. RTDCS отслеживает состояние оборудования, ход выполнения операций и другие параметры в режиме реального времени. Система может автоматически изменять расписание в ответ на непредвиденные события, такие как поломки оборудования, остановки производства и прочие факторы.

Основные преимущества системы:

- Позволяет оперативно реагировать на изменения в производственной среде, снижая потери времени и ресурсов.
- Автоматическая коррекция расписаний помогает избежать простоев и повышает эффективность использования оборудования.

Недостатки:

- Внедрение RTDCS требует значительных финансовых вложений и технического сопровождения.
- Система требует регулярного обновления и мониторинга для поддержания её работоспособности.

3. Системы оперативного управления и диспетчеризации

Современные системы диспетчеризации (MES, BPMS, Workflow) обеспечивают оперативное управление производственными процессами в реальном времени, интегрируя данные с оборудования и информационных

систем. Они позволяют мониторить состояние производства, управлять ресурсами, корректировать планы и прогнозировать сбои.

Интеллектуальные системы диспетчеризации используют методы машинного обучения и анализа больших данных для адаптивного планирования и принятия решений в условиях неопределенности [4].

Основные преимущества:

- Значительно повышают прозрачность и управляемость производства;
- Возможность интеграции с промышленным интернетом вещей (IIoT);

Недостатки:

- Высокая сложность внедрения и интеграции в производство;
- Системы требовательны к качеству и полноте данных;
- Высокие затраты на внедрение и сопровождение.

Данные методы успешно применяются в различных производственных и военных отраслях. Каждый из этих методов имеет свои сильные и слабые стороны и предназначен в основном для полного цикла производства. Задача оптимизации планирования линии покраски пластиковых деталей является более узкоспециализированной и требует учесть множество ограничений и условий. Важно разработать систему планирования с учетом всех требований и технологических возможностей работы линии, приняв во внимание уже существующие решения в данной области.

2.3. Теория расписаний. Выбор методов и алгоритмов для планирования расписания линии окраски

Теория расписаний – это раздел исследования операций, в котором строятся и анализируются математические модели календарного планирования (т.е. упорядочивания во времени) различных целенаправленных действий с учетом целевой функции и различных ограничений [5].

Содержательно многие задачи ТР являются оптимизационными, т.е. состоят в выборе (нахождении) среди множества допустимых расписаний (расписаний, допускаемых условиями задачи) тех решений, на которых достигается “оптимальное” значение целевой функции. Обычно под “оптимальностью” понимается минимальное или максимальное значение некоторой целевой функции.

Задачу данной работы можно классифицировать как циклический поточно-транспортный процесс с зависимым от последовательности временем переналадки (Cyclic Flow Shop Scheduling Problem with Sequence-Dependent Setup Times) с особенностью партийной обработки (batch processing). Эта классификация относится к узкому классу задач теории расписаний, характерному для поточных линий с фиксированным циклом обработки партиями, а основная оптимизация состоит в том, чтобы минимизировать суммарное время переналадки (смены цветов) между партиями и повысить эффективность линии. Данная задача типична для автомобильных производств. Основные характеристики:

1. Циклический характер производства (Cyclic Flow Shop Scheduling Problem): линия окраски функционирует непрерывно в течении смены по круговому маршруту, где каждая тележка проходит полный цикл окраски за фиксированное время, после чего на нее устанавливается новая заготовка;
2. Зависимость переналадки от последовательности (Sequence-Dependent Setup): при смене цвета окраски требуется пропускать 1-2 пустые тележки

для нейтрализации опыла, что представляет собой время переналадки, зависящее от последовательности обработки (цветов);

3. Ограничения на назначения ресурсов: каждая делать крепится на определенный тип тележек с фиксированным назначением, что добавляет ограничение ресурсного характера;
4. Окраска происходи партиям (batch processing): окрашивается группа из примерно 13 тележек одним цветом, включающем все типы деталей согласно плану линии, смена цвета происходит только между партиями разных цветов;
5. Синхронизация со сборкой кузовов: в «онлайн» формате работы готовые детали сразу поступают на сборку кузовов в соответствии с очередью по цветам

Нотация Грэхема – стандартный способ формализовать задачи планирования производства в виде трёх полей: $\alpha \mid \beta \mid \gamma$ (описание производственной среды | описание ограничений и особенностей задачи | критерий оптимизации) [6].

В соответствии с трехполевой нотацией Грэхема данную задачу можно формализовать как:

$$F_{cyclic}^{batch} \mid s_{ij}^{batch}, ficed - assignment, batch - processing \mid min \sum S_{ij}, (1)$$

где:

F_{cyclic}^{batch} – циклическая линия с партийной обработкой;

s_{ij}^{batch} – времена переналадки между партиями;

$ficed - assignment$ – фиксированное назначение типов деталей к типам тележек;

$batch - processing$ – обработка идет партиями;

$min \sum S_{ij}$ – минимизация суммарного времени переналадок между партиями (смен цвета);

2.3.1 Математическая формализация модели

Параметры:

- $P = \{1, 2, \ldots, m\}$ – множество партий (например, $m \approx 5$, если 64 тележки делятся на партии по ~13 тележек);
- $\mathrm{C} = \{\mathrm{c}_1, \ldots, \mathrm{c}_5\}$ – множество цветов;
- Для каждой партии p задан цвет $\mathrm{c}_p \in C$ и фиксированный набор тележек $T_p \subseteq T$ (с учетом типов деталей);
- Время полного цикла покраски: p_{cycle} (фиксировано)
- Время переналадки между цветами партий (пропуск пустых тележек):

$$s(\mathrm{c}_i, \mathrm{c}_j) = \begin{cases} 0, & \mathrm{c}_i = \mathrm{c}_j \\ 1 \cdot p_{cycle}, \mathrm{c}_i \neq \mathrm{c}_j \text{ (пропуск 1 тележки)} \\ 2 \cdot p_{cycle}, \mathrm{c}_i \neq \mathrm{c}_j \text{ (пропуск 2 тележек)} \end{cases}, \quad (2)$$

Переменные:

Последовательность партий: $n = (n_1, n_2, \ldots, n_m)$ – перестановка партий, определяющих порядок их окраски;

Целевая функция – минимизировать суммарное время переналадки между партиями:

$$\min_n \sum_{k=1}^{P-1} s(\mathrm{c}_{n_k}, \mathrm{c}_{n_{k+1}}), \quad (3)$$

(при необходимости можно добавить перенос от последней партии к первой для цикличности)

Ограничения:

- Партии фиксированы по составу (набор тележек и типы деталей);
- Все партии должны быть окрашены до конца смены;

2.3.2 Применимые методы и алгоритмы для решения задачи

Для эффективного планирования и диспетчеризации линии окраски пластиковых деталей необходимо использовать комплексный набор методов. Нужно учитывать как классические задачи теории расписаний с

фиксированными параметрами, так и динамические изменения производственного плана, а также возможность быстрой адаптации расписания при изменении условий.

Классические методы планирования:

1. Целочисленное линейное программирование (ILP) [7]

Мощный инструмент для точного решения задач расписаний с фиксированными параметрами. ILP позволяет оптимизировать порядок партой по цветам с минимизацией суммарного времени переналадки, учитывая фиксированное время цикла и назначение типов деталей по тележкам.

2. Эвристика и метаэвристика (Генетические алгоритмы, LNS)

Для больших задач ILP становится вычислительно долгим, поэтому применяются эвристические алгоритмы для быстрого нахождения хорошего решения:

- Генетические алгоритмы (ГА):

Представляют расписание как хромосому (последовательность партий). Используют операторы скрещивания и мутации для поиска оптимального решения. Функция пригодности учитывает суммарное время переналадки и штрафы (понижение приоритета выполнения) за нарушение ограничений [8]:

$$f(n) = \sum_{k=1}^{m-1} s\left(\mathrm{c}_{n_k}, \mathrm{c}_{n_{k+1}}\right) + \lambda \cdot \text{штрафы}\,, \tag{4}$$

- Поиск с большими окрестностями (Large Neighborhood Search, LNS) [9]:

Итеративно изменяет часть решения (например, переставляет несколько партий), используя точные или эвристические методы для восстановления решения

Методы для адаптивного и оперативного планирования:

1. Стохастическое программирование с несколькими сценариями (Scenario-based Stochastic Programming):

Учитывает неопределенность спроса и изменений очереди сборки, моделируя несколько сценариев развития событий. Для каждого сценария

строится оптимальное расписание, затем выбирается решение, оптимальное по среднему или худшему сценария [10].

$$min \sum_{s=1}^{S} p_s \sum_{t=1}^{P} s\left(c_{s,t}, c_{s,t+1}\right), \tag{5}$$

где p_s – вероятность сценария s.

Данный метод позволяет планировать смены с учетом вероятных изменений спроса и повышать устойчивость расписания.

2. Стохастическое динамическое программирование (SDP):

Оптимизирует последовательность решений с учетом вероятностных переходом между состояниями. Позволяет корректировать расписание по мере поступления новой информации [11].

Уравнение Беллмана:

$$V(s) = \min_{a} \left[c(s,a) + \gamma \sum_{s'} P(s'|s,a) V(s') \right], \tag{6}$$

где s – состояние, a – действие, P – вероятность перехода.

Данный метод позволяет осуществлять быстрое планирование при изменениях очереди сборки и управлять рисками и непредвиденными событиями.

3. Планирование с подвижным горизонтом (Rolling Horizon Planning)

План строится на ограниченный горизонт времени и регулярно пересчитывается при поступлении новых данных, что позволяет адаптировать расписание к изменениям, сохраняя стабильность. Используется в сочетании с ILP и эвристиками.

Планируемая структура программной среды:

В основном планирование будет вестись по фиксированным данным, которые будут обновлять только два раза за смену, что позволяет поставить в основе программы классическую задачу расписания. При наступлении определенных событий (обновления очереди или остановок линии) будет

использоваться корректировка с помощью сценарного анализа и стохастического программирования. Получаем следующую структуру программы, сочетающую в себе гибридный подход:

1. Модуль базового планирования:
    - ILP или эвристики для построения основного расписания на смену;
    - Учет текущих данных по складу и очереди сборки кузовов;
2. Модуль сценарного анализа и стохастического планирования:
    - Генерация сценария спроса и оценка устойчивости расписания;
    - Просмотр альтернативных планов;
3. Модуль адаптивного планирования (опционально):
    - Обработка изменения очереди сборки кузовов (2 раза за смену);
    - Быстро корректирует расписание с помощью эвристик и динамического программирования.

Данная структура обеспечивает баланс между качеством расписания и скоростью перерасчета, что критично для эффективной работы линии окраски. В будущем при появлении необходимости более глубокого анализа влияния случайных событий, тестирования новых стратегий и выявления скрытых проблем, можно добавить имитационное моделирование линии окраски, что позволит моделировать производственный процесс и тестировать различные подходы планирования.

2.4. Выбор языка программирования

При разработке системы планирования для линии окраски пластиковых деталей ключевым вопросом является выбор формы реализации: специализированный программный код или полноценное приложение с графическим интерфейсом (GUI). Каждый подход имеет свои преимущества и ограничения, которые необходимо проанализировать и сопоставить с задачей дипломной работы, чтобы выбрать оптимальный вариант реализации на первоначальном этапе.

Аргументы в пользу программного кода:

- Фокус на алгоритмической составляющей: написание кода позволяет сосредоточиться на логике решения задачи, избежав затрат времени на разработку интерфейса;
- Интеграция с Excel: использования различных библиотек обеспечивает прямую работу с входными и выходными данными в табличном формате Excel, что соответствует условиям работы предприятия;
- Гибкость и скорость разработки: код легко модифицируется для тестирования различных алгоритмов, что важно для научного исследования и конкретной задачи;

Аргументы в пользу графического приложения:

- Удобство для пользователей: интерактивный интерфейс, визуализация расписание, удобное редактирование данных позволяет любому сотруднику быстро разобраться в работе программы без должных навыков программирования;
- Расширенные функциональные возможности: в приложение можно добавить симуляцию работы линии, автоматическую генерацию отчетов, управление исключениями, что сделает программу более обширной и позволит затронуть сразу несколько важных областей работы;

Так как основная цель данной работы – продемонстрировать алгоритм планирования, а не создать промышленное ПО, на текущий момент работа с программным кодом является более обоснованной, так как позволяет реализовать основную логику алгоритма, быстро и удобно работать с таблицами и использовать оптимизационные библиотеки. При выборе графического приложения придется значительно увеличить сроки разработки, учесть более высокие требования к ресурсам и сложность внедрения нового ПО в производство, однако GUI можно принять за перспективу дальнейшей разработки.

При выборе языка программирования рассмотрено несколько основных языков, которые активно используются на производстве на данный момент. В качестве основного языка программирования был выбран Python по следующим причинам [12]:

- Богатая экосистемы библиотек: такие библиотеки как «pandas», «ortools», «openpyxl» и другие обеспечивают чтение и запись Excel-таблиц, предоставляют инструменты для решения задач оптимизации, включая линейное программирование и алгоритмы теории расписаний, что необходимо для решения поставленной задачи;
- Простота и скорость разработки: динамическая типизация и лаконичный синтаксис Python сокращает время написания кода и упрощает отладку;
- Производительность: хотя Python уступает компилируемым языкам (C++, Java) в скорости выполнения, для данной задачи это некритично. Алгоритм работает в оффлайн-режиме, а время расчета расписания для тележек измеряется секундами.
- Академическая популярность: Python широко используется в научных исследованиях, что подтверждается наличием готовых решений для аналогичных задач;

Альтернативные языки и их ограничения:

- Java / C#: строгая типизация и необходимость явного объявления классов усложняют прототипирование, интеграция с Excel требует дополнительных библиотек, что увеличивает сложность проекта:
- R / Matlab: специализированы на статистике и математическом моделировании, но недостаточно гибкие для работы с производственными ограничениями;
- VBA: интегрирован в Excel, но его использование ограничено из-за слабой поддержки современных алгоритмов оптимизации;

Выбор Python в сочетании с библиотеками для оптимизации и работы с данными позволяет реализовать алгоритм, минимизирующий простои и смену цветов, обеспечить прозрачность расчетов, быстро адаптировать решение под

изменяющиеся требования. Этот подход соответствует критериям эффективности, простоты внедрения и научной обоснованности, что делает его хорошим выбором для выполнения поставленной задачи.

3. Техническое задание

3.1. Общие сведения

Полное наименование системы и ее условное обозначение:

Система планирования и диспетчеризации линии окраски пластиковых деталей

Наименование предприятий (объединений) разработчика и заказчика (пользователя) системы и их реквизиты:

Разработчик:

Максимов Дмитрий Игоревич

Предприятие:

АО МАЗ «Москвич»

Заказчик: АО МАЗ «Москвич»

Юридический адрес: г. Москва, Волгоградский проспект, д. 42, к. 36.

Плановые сроки работы по созданию системы

Дата начала работы – 01.09.2024 г.

Дата окончания работы – 30.05.2025 г.

3.2. Назначение и цели создания системы

Назначение системы:

Разрабатываемая система предназначена для планирования и диспетчеризации линии окраски пластиковых деталей.

Цели создания системы:

Система планирования и диспетчеризации линии окраски пластиковых деталей позволяет решать следующие задачи:

- Повышение эффективности производства за счет минимизации простоев оборудования, сокращение времени выполнения плана и повышении объема производимых деталей;

- Составление и обновление плана окрашивания, учитывая факторы загрузки оборудования, временные ограничения и технологические особенности процесса окраски;
- Снижение вероятности ошибок благодаря сокращению влияния человеческого фактор.

3.3. Требования к системе в целом

- Планирование, основанное на фильме и удовлетворяющее очередь сборки кузовов;
- Минимизация простоев оборудования и внеплановых остановок производства;
- Совместимость с существующим оборудованием и программным обеспечением;
- Возможность обновления плана в зависимости от изменений фильма;
- Учет технологических параметров и ограничений линии окраски;

Техническое задание разработано в соответствии с ГОСТ 34.602-89 [13].

4. Структурное проектирование

4.1. Разработка блок-схемы основных алгоритмов работы программы

Для описания последовательности работы программы планирования покраски на смену разработаем блок-схему и опишем ее основные алгоритмы [14].

При запуске программы загружаются следующие **входные данные**:

- Запасы покрашенных деталей по типам (бампер, спойлер, транклид) и цветам (5 цветов) – таблица Excel (из 1С) или ручной ввод;
- Данные об очереди сборки кузовов (фильм) – Excel-таблица м цветами и порядком сборки (обновляется 2 раза в смену);
- Технологические ограничения: параметры партий (4 партии по 13 тележек, 1 партия на 12 тележек), время покраски одной партии (фиксированное), время максимального ожидания партии, эффективность линии – ручной ввод;

На **выходе** мы получается Excel-таблицу с очередью покраски пластиковых деталей, в которой будут отражены следующие данные: порядковый номер партии, цвет партии, типы деталей по тележкам, планируемое время начала и окончания покраски.

Основные этапы алгоритма работы программы планирования:

1) **Загрузка и проверка данных:** подсчет данных запасов и очереди сборки кузовов, ручной ввод технологических параметров через форму или конфигурационный файл, проверка корректности введенных данных;
2) **Формирования списка партий покраски:** формирование партий на смену по цветам и типам деталей на основе очереди сборки и имеющихся запасов;
3) **Оптимизация порядка партий:** применение ILP и эвристических алгоритмов для минимизации суммарного времени переналадки (смены цветов) между партиями, учитывая, что между сменой цвета нужно

пропускать 1-2 пустые тележки (время переналадки), выбор наилучшего варианта расписания;

4) **Расчет расписания:** вычисление времени начала и окончания покраски каждой партии с учетом фиксированного времени цикла и эффективности линии, обязательная проверка, что время ожидании партии перед отправкой на сборку не превышает допустимое (нужный комплект деталей должен отправлять на сборку до прихода кузова);
5) **Генерация выходного файла:** создание и сохранение Excel-таблицы с расписанием покраски;
6) **Завершение работы программы**

Блок-схема работы программы представлена на рисунке 9.

Рисунок 9 – Алгоритм работы программы

5. Рабочее проектирование

5.1. Разработка программного кода

На начальном этапе разработки программы необходимо определить набор требуемых библиотек, а также инициализировать и загрузить вводные данные. Для комфортной работы программы понадобятся следующие библиотеки [15]:

- «pandas» – для работы с табличными данными (запасы на складе, очередь покраски кузовов);
- «openpyxl» – для чтения/записи Excel-таблиц (совместимость с современными форматами);
- «configparser» – для гибкой настройки параметров через файл config.ini (удобный способ централизованного хранения настроек и параметров программы вне основного кода);
- «pulp» – для реализации ILP-оптимизации;
- «random» – функции для генерации случайных чисел и работы со случайным выбором элементов из последовательностей;
- «typing» – для лучшей читаемости кода через аннотации типов;
- «math» – для математических уравнений и функций;
- «deap» – для реализации генетических алгоритмов;
- «deaptime» – для форматирования и работы с временными интервалами;

Для хранения настроек и параметров работы программы (технологических характеристик линии и ссылок на файлы с Excel-таблицами) будет использоваться конфигурационный файл, что позволит легко менять настройки без изменения кода программы (листинг 1).

Далее загружаем данные о запасах покрашенных деталей на складе, таблицу очереди сборки кузовов (фильм), поступающую из цеха сборки в начале смены, и технологические параметры (время покраски партии, эффективность работы линии и время переналадки (смены цвета)). Программа проверяет

обозначения и структуру файлов, также фильтрует очередь сборки кузовов на нужную дату (указывает пользователь).

Следующий этап – формирование списка партий покраски на смену. Программа подсчитывает количество деталей каждого цвета и тип необходимых для сборки на смену и количество готовых покрашенных деталей, хранящихся на складе к началу смены.

```
[PATHS]
stocks_file = D:/VKR/Stocks.xlsx
queue_file = D:/VKR/Film.xlsx
output_file = D:/VKR/Plan.xlsx
[TECHNOLOGY]
time_per_batch = 101
efficiency = 0.94
setup_time = 4.7
```

Листинг 1 – Конфигурационный файл

Учитывая запас, подсчитываем детали, которые нужно покрасить за смену, чтобы полностью удовлетворить план сборки кузовов (листинг 2):

```
def calc_to_paint(required: Dict[str, Dict[str, int]], stock:
Dict[str, Dict[str, int]]) -> Dict[str, Dict[str, int]]:
  to_paint = {}
  for color in required:
    to_paint[color] = {}
    for part in required[color]:
      stock_qty = stock.get(color, {}).get(part, 0)
      need_qty = required[color][part] - stock_qty
      to_paint[color][part] = max(need_qty, 0)
  return to_paint
```

Листинг 2 – Подсчет деталей необходимых к покраске (учитывая запас)

Далее программа формирует партии деталей на смену по цветам и типам деталей, принимая в расчет фиксированное количество и назначение тележек, порядок партий и цикличность линии. Если запасов хватает, партия будет формироваться с пустыми тележками, которые мы сможем использовать в дальнейшем для переналадки. При возможности покраска деталей будет происходить с переизбытком (пустые тележки будут заполняться на усмотрение операторов линии), чтобы пополнять складские запасы на следующую смену. На

листинге 3 продемонстрирован цикл формирования партий, который является заключительной частью функции формирования партий.

```
   while any(c['remaining'] > 0 for c in color_queue):
     for batch_size in batch_sizes:
         next_color_entry = next((c for c in color_queue if
c['remaining'] > 0), None)
         if next_color_entry is None:
             break
         color = next_color_entry['color']
         parts = to_paint[color]
         batch = {
           'Партия №': batch_number,
           'Цвет': color,
           'Размер партии': batch_size,
           'Бампер передний': min(parts['Бампер передний'] -
(color_batches_needed[color] - next_color_entry['remaining']) *
batch_capacity['Бампер передний'], batch_capacity['Бампер
передний']),
           'Бампер задний': min(parts['Бампер задний'] -
(color_batches_needed[color] - next_color_entry['remaining']) *
batch_capacity['Бампер задний'], batch_capacity['Бампер
задний']),
           'Спойлер': min(parts['Спойлер'] -
(color_batches_needed[color] - next_color_entry['remaining']) *
batch_capacity['Спойлер'], batch_capacity['Спойлер']),
           'Транклид': min(parts['Транклид'] -
(color_batches_needed[color] -next_color_entry['remaining']) *
batch_capacity['Транклид'], batch_capacity['Транклид']),
             'Тип тележек': trolley_counts}
       for key in ['Бампер передний', 'Бампер задний',
'Спойлер', 'Транклид']:
             if batch[key] < 0:
                  batch[key] = 0
       batches.append(batch)
       batch_number += 1
       next_color_entry['remaining'] -= 1
       if all(c['remaining'] <= 0 for c in color_queue):
                break
    return batches
```

Листинг 3 – Цикл формирование партий для покраски

В результате выполнения программа выводит список сформированных партий покраски на смену, присваивая каждой партии свой номер и цвет, а также информацию о типе партии и заполнении тележек по типам деталей.

Следующий этап – оптимизация порядка партий. Важно учесть, что порядок покраски партий должен удовлетворять очередь сборки кузовов, чтобы

детали нужных цветов всегда были в наличии до прихода кузовов на сборку. Основная задача – минимизировать переналадки (смену цветов).

Сначала задаются функции получения приоритетов цветов из очереди сборки кузовов, оценки порядка партий с учетом приоритетов и расписания и проверти своевременности расписания. В данных функциях вводятся штрафы за нарушения порядка приоритетов, опоздание деталей на сборку и смену цвета (переналадки). Данные штрафы позволяют определять программе, какие партии деталей важно запустить раньше, в зависимости от запасов покрашенных деталей на складе и очереди сборки кузовов.

Далее следует оптимизация порядка партий. В ходе написания программы на данном этапе было принято решение отказаться от целочисленного линейного программирования, так как ILP требует высоких вычислительных мощностей и много времени для поиска оптимального решения, что критично для быстрого составления расписания на персональном компьютере. Однако в перспективе программа позволяет дополнять и вводить новые алгоритмы оптимизации очереди покраски для сравнения различных результатов и выбора оптимального решения. Для прототипа программы основным методом оптимизации является более быстрый генетический алгоритм (ГА) на базе DEAP [16], обеспечивающий глобальный поиск с помощью эволюции популяции (листинг 4):

```
def setup_genetic_algorithm(batches: List[Dict], queue_df:
pd.DataFrame, stock, to_paint):
    creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
    creator.create("Individual", list,
fitness=creator.FitnessMin)

    toolbox = base.Toolbox()

    def init_heuristic():
        color_priorities = get_color_priorities(queue_df)
        color_to_batches = defaultdict(list)
        for idx, batch in enumerate(batches):
            color_to_batches[batch['Цвет']].append(idx)
        sorted_colors = sorted(color_to_batches.keys(),
key=lambda c: color_priorities.get(c, 10000))
        sorted_indices = []
        for color in sorted_colors:
            sorted_indices.extend(color_to_batches[color])
        return sorted_indices
    def group_mutation(individual):
      color_groups = defaultdict(list)
      for idx in individual:
        color = batches[idx]['Цвет']
        color_groups[color].append(idx)

      # Перемешиваем порядок цветов, но сохраняем группы
      colors = list(color_groups.keys())
      random.shuffle(colors)
      new_individual = []
      for color in colors:
          new_individual.extend(color_groups[color])

      return new_individual,

    toolbox.register("init_heuristic", init_heuristic)
    toolbox.register("individual", tools.initIterate,
creator.Individual, toolbox.init_heuristic)
    toolbox.register("population", tools.initRepeat, list,
toolbox.individual)
    toolbox.register("group_mutate", group_mutation)
    toolbox.register("evaluate",
functools.partial(eval_individual, batches=batches,
queue_df=queue_df, stock=stock, to_paint=to_paint))
    toolbox.register("mate", tools.cxOrdered)
    toolbox.register("mutate", tools.mutShuffleIndexes,
indpb=0.2)
    toolbox.register("select", tools.selTournament, tournsize=5)

    return toolbox
```

Листинг 4 – Генетический алгоритм (DEAP)

В паре с ГА будет работать поиск в больших окрестностях (Large Neighborhood Search (LNS)) – метод локального поиска, который улучшает качество найденного решения, изменяя одновременно большую часть очереди и делая поиск более глубоким и эффективным (листинг 5). Вместе эти методы дают баланс между исследованием пространства решений и детальной оптимизацией порядка партий.

В результате выполнения данного этапа программа выводит оптимальное расписание партий (с указанием цветов), рассчитывая время начала и окончания покраски каждой партии. Также пользователь получает информацию, между какими партиями необходимо пропустить 2 пустые тележки при смене цвета.

```
def lns_local_search(sequence, batches, queue_df, stock,
to_paint, max_iter, destroy_size):
    def evaluate(seq):
        return evaluate_sequence_with_priority(seq, batches,
queue_df, stock, to_paint,)

    best_seq = sequence[:]
    best_score = evaluate(best_seq)

    for iteration in range(max_iter):
        start = random.randint(0, len(sequence) - destroy_size)
        end = start + destroy_size

        destroyed = best_seq[start:end]
        remainder = best_seq[:start] + best_seq[end:]

        random.shuffle(destroyed)
        new_seq = remainder[:start] + destroyed +
remainder[start:]

        new_score = evaluate(new_seq)
        if new_score < best_score:
            best_seq = new_seq
            best_score = new_score
            print(f"LNS улучшил результат на итерации
{iteration}, штраф: {best_score}")

    return best_seq
```

Листинг 5 – LNS для локального улучшения

Заключительный этап – программа генерирует выходной файл в виде Excel-таблицы, куда загружает оптимизированную очередь покраски. Полученный файл сохраняется по пути, указанном в конфигурационном файле.

6. Апробирование

6.1. Тестирование работы программы

Для проведения тестирования работы программы были использованы файлы и таблицы, предоставленные заводом «Москвич» для исследования и выполнения дипломной работы. Для проведения опыта в расписании сборки кузовов (фильме) выбрана смена за 17.12.2024 (рисунок 10). По плану в этот день на сборку поступило 126 кузовов разных цветов:

ФИЛЬМ DKD +															балкон =19
Дата входа на сборку M131	Номер по порядку	VIN	Тара с кузовом	Модель	Комплектация	Тип	Цвет	Жижа	Мотор	Номер мотора	тип	ТСМ +	прогноз ТСМ	спуск на сборку с балкона
17.12.2024	1	X7LJA15BAC1020816	CKD	M3	Flagship	CVT	Голубой + Черн	СУХОЙ			CKD	#Н/Д		17
17.12.2024	2	X7LJA13BAC1019148	CKD	M3	Luxury heated	CVT	Красный	СУХОЙ			CKD	#Н/Д		17
17.12.2024	3	X7LJA13BAC1020302	CKD	M3	Luxury heated	CVT	Красный	СУХОЙ			CKD	#Н/Д		17
17.12.2024	4	X7LJA13BAC1015353	CKD	M3	Luxury heated	CVT	Красный	СУХОЙ			CKD	#Н/Д		17
17.12.2024	5	X7LJA13BAM1019311	CKD	M3	Luxury heated	MT	Черный	СУХОЙ			CKD	#Н/Д		17
17.12.2024	6	X7LJA13BAC1020254	CKD	M3	Luxury heated	CVT	Красный	СУХОЙ			CKD	#Н/Д		17
17.12.2024	7	X7LJA13BAM1019316	CKD	M3	Luxury heated	MT	Черный	СУХОЙ			CKD	#Н/Д		17
17.12.2024	8	X7LJA13BAC1020182	CKD	M3	Luxury heated	CVT	Красный	СУХОЙ			CKD	#Н/Д		17
17.12.2024	9	X7LJA13BAM1019329	CKD	M3	Luxury heated	MT	Черный	СУХОЙ			CKD	#Н/Д		17
17.12.2024	10	X7LJA13BAC1020341	CKD	M3	Luxury heated	CVT	Красный	СУХОЙ			CKD	#Н/Д		17
17.12.2024	11	X7LJA13BAC1020205	CKD	M3	Luxury heated	CVT	Красный	СУХОЙ			CKD	#Н/Д		17
17.12.2024	12	X7LJA13BAC1020344	CKD	M3	Luxury heated	CVT	Красный	СУХОЙ			CKD	#Н/Д		17
17.12.2024	13	X7LJA15BAC1020635	CKD	M3	Flagship	CVT	Красный + Черн	СУХОЙ			CKD	#Н/Д		17
17.12.2024	14	X7LJA13BAC1019826	CKD	M3	Luxury heated	CVT	Красный	СУХОЙ			CKD	#Н/Д		17
17.12.2024	15	X7LJA13BAC1020211	CKD	M3	Luxury heated	CVT	Красный	СУХОЙ			CKD	#Н/Д		17
17.12.2024	16	X7LJA13BAM1018708	CKD	M3	Luxury heated	MT	Серый	СУХОЙ			CKD	#Н/Д		17
17.12.2024	17	X7LJA13BAC1020277	CKD	M3	Luxury heated	CVT	Красный	СУХОЙ			CKD	#Н/Д		17
17.12.2024	18	X7LJA13BAM1016107	CKD	M3	Luxury heated	MT	Серый	СУХОЙ			CKD	#Н/Д		17

Рисунок 10 – Пример фильма сборки кузовов для тестирования

Данные о запасах покрашенных деталей на складе были предоставлены только в виде скриншотов, поэтому для проведения тестирования была создана Excel-таблица, симулирующая состояние склада к началу смены (таблица 2):

Таблица 2 – Запасы покрашенных деталей на складе

Номенклатура	Цвет	Кол-во на складе
КОРПУС ПЕРЕДНЕГО БАМПЕРА Белый	WH1-0	3
КОРПУС ПЕРЕДНЕГО БАМПЕРА Голубой	LB1-0	3
КОРПУС ПЕРЕДНЕГО БАМПЕРА Красный	RE1-0	3
КОРПУС ПЕРЕДНЕГО БАМПЕРА Серый	GR1-0	3
КОРПУС ПЕРЕДНЕГО БАМПЕРА Черный	BK1-0	3
КОРПУС ЗАДНЕГО БАМПЕРА Белый	WH1-0	3
КОРПУС ЗАДНЕГО БАМПЕРА Голубой	LB1-0	3
КОРПУС ЗАДНЕГО БАМПЕРА Красный	RE1-0	3
КОРПУС ЗАДНЕГО БАМПЕРА Серый	GR1-0	3
КОРПУС ЗАДНЕГО БАМПЕРА Черный	BK1-0	3
СПОЙЛЕР Белый	WH1-0	6
СПОЙЛЕР Голубой	LB1-0	6
СПОЙЛЕР Красный	RE1-0	6
СПОЙЛЕР Серый	GR1-0	6
СПОЙЛЕР Черный	BK1-0	6
ТРАНКЛИД Белый	WH1-0	6
ТРАНКЛИД Голубой	LB1-0	6
ТРАНКЛИД Красный	RE1-0	6
ТРАНКЛИД Серый	GR1-0	6
ТРАНКЛИД Черный	BK1-0	6

Выполнение программы занимает около 2 минут. Время зависит от параметров алгоритмов оптимизации партии, которые можно настраивать в зависимости от требований. В результате запуска было сформировано 19 партий, которые были оптимизированы и объединены в единую очередь покраски. На выходе получается удобная Excel-таблица, в которой отображена вся необходимая информация для каждой партии деталей: номер очереди покраски, цвет партии, количество деталей каждого типа, время начала и окончания

покраски, а также информация о необходимости переналадки (смены цвета) и пропуска пустых тележек (таблица 3):

Таблица 3 – Результат тестирования программы

Порядок в плане	Цвет	Передних бамперов	Задних бамперов	Спойлеров	Транклидов	Размер партии (тележек)	Время начала (чч:мм)	Время окончания	Пустых тележек	Смена цвета
1	Голубой	3	3	0	0	13	00:00	01:41	0	Нет
2	Красный	7	7	6	9	13	01:41	02:02	2	Да
3	Красный	7	7	6	5	13	02:02	02:22	0	Нет
4	Черный	7	7	6	9	13	02:22	02:43	2	Да
5	Красный	7	7	6	9	13	02:43	03:03	2	Да
6	Красный	7	7	6	9	13	03:03	03:24	0	Нет
7	Красный	4	4	6	0	13	03:24	03:44	0	Нет
8	Красный	0	0	2	0	13	03:44	04:05	0	Нет
9	Красный	7	7	6	9	13	04:05	04:25	0	Нет
10	Красный	7	7	6	0	13	04:25	04:46	0	Нет
11	Серый	7	7	6	9	13	04:46	05:06	2	Да
12	Серый	7	7	6	9	13	05:06	05:27	0	Нет
13	Красный	7	7	6	9	13	05:27	05:47	2	Да
14	Серый	7	7	6	9	13	05:47	06:08	2	Да
15	Черный	6	6	4	1	13	06:08	06:29	2	Да
16	Серый	7	7	6	4	13	06:29	06:49	2	Да
17	Серый	7	7	6	0	13	06:49	07:10	0	Нет
18	Серый	1	1	4	0	13	07:10	07:30	0	Нет
19	Серый	7	7	6	9	13	07:30	07:51	0	Нет

Важно заметить, что программа создана для «онлайн» режима работы линии и заполняет партии деталями только так, чтобы покрыть необходимый

минимум. Образованные пустые тележки операторы линии могут использовать для переналадки, либо укомплектовывать партии, чтобы пополнять складские запасы.

Полученное решение не всегда будет 100% оптимальным расписанием, что можно заметить по полученным результатам. Теория расписания не может гарантировать нахождение наилучшего решения для сложных производственных задач с множеством ограничений. Однако примененные алгоритмы позволяют находить расписания, приближенные к оптимуму. Сгенерированная таблица с конечным расписанием покраски предоставляет операторам удобный план, позволяя легко вносить корректировки и менять партии местами при необходимости.

ЗАКЛЮЧЕНИЕ

В ходе выполнения квалификационной работы были выполнены следующие **задачи**:

1) Проведен анализ принципа работы, планирования и диспетчеризации линии окраски пластиковых деталей АО МАЗ «Москвич»;
2) Проведен анализ современных методов планирования и диспетчеризации производственных процессов;
3) Проведен анализ основных алгоритмов и методов теории расписания, которые применяются для планирования производств;
4) Разработан алгоритм планирования с учетом приоритетов и производственных ограничений, сочетающий в себе генетический алгоритм и методы локального поиска;
5) Реализован программный код, позволяющий автоматизировать процесс планирования и составления расписания на смену;
6) Проведено тестирование разработанной системы на реальных производственных данных завода АО МАЗ «Москвич»;

По итогам проделанной работы и проведенного тестирования можно подтвердить эффективность предложенного решения. Разработанный алгоритм позволяет значительно повысить эффективность и скорость планирования, снизить количество переналадок и улучшить своевременность поставки деталей на сборку, что способствует повышению производительности и конкурентоспособности предприятия.

Однако используемые методы оптимизации, основанные на вероятностных и эвристических алгоритмах, не дают гарантию нахождения 100% оптимального решения для сложной производственной задачи. Тем не менее, разработанная система обладает высоким потенциалом для поиска качественных решений в быстрые сроки, что делает ее практично применимой и полезной в условиях реального производства с множеством ограничений и динамических факторов.

Рекомендации и разработанный инструмент могут быть использованы для дальнейшего совершенствования систем планирования и управления производством.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1) Техническая документация АО МАЗ “Москвич”
2) Хуссейн Л.А. Оценка использования систем планирования ресурсов российскими предприятиями — Московский экономический журнал, 2022, № 2.
3) Умнов Е.А., Умнов А.Е. Задача математического программирования для комплекса математических моделей – Москва: Труды МФТИ, Том 9, №4, 2017 – с.149 – 159
4) Рыжко А.Л., Рыбников А.И., Рыжко Н.А. Информационные системы управления производственной компанией : учебник для вузов – Москва : Издательство Юрайт, 2025. – 354 с.
5) Лазарев А.А. Теория расписаний. Методы и алгоритмы – М.: ИПУ РАН, 2019 – 408 с.
6) Аничкин А.С., Семенов В.А. Современные модели и методы теории расписаний и календарно-сетевого планирования – Труды Института системного программирования РАН, Т. 26. Вып. 3, 2014. – с. 212 – 262.
7) Заозерская Л.А., Ильев В.П., Леванова Т. В. Методы оптимизации: целочисленное линейное программирование: учебное пособие – Омск : Омский государственный университет им. Ф. М. Достоевского (ОмГУ), 2020. – 40 с.
8) Вирсански Э. Генетические алгоритмы на Python: руководство; перевод с английского А. А. Слинкина. – Москва : ДМК Пресс, 2020. — 286 с.
9) Winter F., Musliu N. A large neighborhood search approach for the paint shop scheduling problem. – Journal of Scheduling 25, 2022. – p. 453 – 475
10) Розанова Л.Ф., Розанова Ж.Б., Чендулаева К.Б. Стохастическое программирование в задачах планирования на предприятиях с непрерывным процессом производства – Управление экономическими системами: электронный журнал, 2014. – 1-9 с.

11) Карпенко А.П. Современные алгоритмы поисковой оптимизации. Алгоритмы, вдохновленные природой: учебное пособие – 3-е изд., испр. – Москва: Издательство МГТУ им. Н. Э. Баумана, 2021. — 446 с.

12) Чернышев С. А. Основы программирования на Python : учебник для вузов – 2-е изд., перераб. и доп. – Москва: Издательство Юрайт, 2025. — 349 с.

13) ГОСТ 34.602-89. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы. – М.: Москва Стандартинформ, 2009 – 12 с.

14) Сандлерман М.Н. Теория Блок-схем – Уральский государственный аграрный университет, 2015. – 8 с.

15) Бизли Д. Python. Подробный справочник, 4-е издание. – Перевод с английского – СПб.: Символ-Плюс, 2010 – 864 с.

16) Официальная документация DEAP. [Электронный ресурс]. – URL: https://deap.readthedocs.io/en/master/index.html (дата обращения 15.05.2025)

ПРИЛОЖЕНИЕ А

Листинг программы:

```
import pandas as pd
from openpyxl import load_workbook
from openpyxl.utils import get_column_letter
from openpyxl.styles import PatternFill, Alignment
import configparser
import pulp
import random
from typing import Dict, List
import math
from datetime import timedelta
from deap import base, creator, tools, algorithms
from collections import defaultdict
import functools

config = configparser.ConfigParser()
config.read("D:/VKR/config.ini", encoding='utf-8')

STOCKS_FILE = config.get('PATHS', 'stocks_file',
fallback='stocks.xlsx')
QUEUE_FILE = config.get('PATHS', 'queue_file',
fallback='queue.xlsx')

color_map_stocks = {'WH1-0': 'Белый','LB1-0': 'Голубой','RE1-0':
'Красный','GR1-0': 'Серый','BK1-0': 'Черный',}

color_map_queue = {
    'Белый': 'Белый', 'Голубой': 'Голубой', 'Красный': 'Красный',
'Серый': 'Серый', 'Черный': 'Черный', 'Голубой + Черная крыша':
'Голубой', 'Красный + Черная крыша': 'Красный',}

def load_stocks(file_path: str) -> pd.DataFrame:
    df = pd.read_excel(
        file_path,
        sheet_name='Stocks',
        usecols=['Номенклатура', 'Цвет', 'Кол-во на складе'],
        dtype={'Кол-во на складе': int}
    )
    df['Цвет'] = df['Цвет'].map(color_map_stocks)

    if df['Цвет'].isnull().any():
        missing = df[df['Цвет'].isnull()]['Цвет'].unique()
        raise ValueError(f"Неизвестные обозначения цветов в
запасах: {missing}")

    required_columns = ['Номенклатура', 'Цвет', 'Кол-во на
складе']
    if not all(col in df.columns for col in required_columns):
```

```
        raise ValueError("Некорректная структура файла запасов")
    return df

def load_queue(file_path: str) -> pd.DataFrame:
    df = pd.read_excel(
        file_path,
        sheet_name='DKD+',
        header=1,
        parse_dates=['Дата входа на сборку M131'],
        usecols=['Дата входа на сборку M131', 'Номер по порядку',
'Цвет']
    )
    df['Цвет'] = df['Цвет'].map(color_map_queue)

    current_date = pd.Timestamp('2024-12-17').normalize()
    df = df[df['Дата входа на сборку M131'] == current_date]
    return df.sort_values('Номер по
порядку').reset_index(drop=True)

time_per_batch = config.getfloat('TECHNOLOGY', 'time_per_batch')
time_btw_telezhka = config.getfloat('TECHNOLOGY',
'time_btw_telezhka')
efficiency = config.getfloat('TECHNOLOGY', 'efficiency')
setup_time = config.getfloat('TECHNOLOGY', 'setup_time')
setup_telezhka_count = config.getfloat('TECHNOLOGY',
'setup_telezhka_count')
assembly_start_delay = config.getfloat('TECHNOLOGY',
'assembly_start_delay')
assembly_interval = config.getfloat('TECHNOLOGY',
'assembly_interval')

stocks_df = load_stocks(STOCKS_FILE)
queue_df = load_queue(QUEUE_FILE)

print("\n Данные успешно загружены!")
print("Статистика запасов:")
print(stocks_df)
print("\n Очередь сборки:")
print(queue_df)
print(f"Время покраски партии: {time_per_batch} мин")
print(f"Эффективность линии: {efficiency}")
print(f"Время переналадки: {setup_time} мин")

def extract_color_from_name(name: str, color_list: List[str]) ->
str:
    for color in color_list:
        if color.lower() in name.lower():
            return color
    return None

def count_required_parts(queue: pd.DataFrame) -> Dict[str,
Dict[str, int]]:
```

```
    required = {}
    for _, row in queue.iterrows():
        color = row['Цвет']
        if color not in required:
            required[color] = {'Бампер передний': 0, 'Бампер
задний': 0, 'Спойлер': 0, 'Транклид': 0}
        required[color]['Бампер передний'] += 1
        required[color]['Бампер задний'] += 1
        required[color]['Спойлер'] += 1
        required[color]['Транклид'] += 1
    return required

def count_stock_parts(stocks: pd.DataFrame) -> Dict[str, Dict[str,
int]]:
    stock = {}
    for _, row in stocks.iterrows():
        name = row['Номенклатура']
        color = extract_color_from_name(name, color_map_queue)
        if color is None:
            continue
        if 'КОРПУС ПЕРЕДНЕГО БАМПЕРА' in name.upper():
            part = 'Бампер передний'
        elif 'КОРПУС ЗАДНЕГО БАМПЕРА' in name.upper():
            part = 'Бампер задний'
        elif 'СПОЙЛЕР' in name.upper():
            part = 'Спойлер'
        elif 'ТРАНКЛИД' in name.upper():
            part = 'Транклид'
        else:
            continue

        qty = row['Кол-во на складе']
        if color not in stock:
            stock[color] = {'Бампер передний':0, 'Бампер
задний':0, 'Спойлер':0, 'Транклид':0}
        stock[color][part] += qty
    return stock

def calc_to_paint(required: Dict[str, Dict[str, int]], stock:
Dict[str, Dict[str, int]]) -> Dict[str, Dict[str, int]]:
    to_paint = {}
    for color in required:
        to_paint[color] = {}
        for part in required[color]:
            stock_qty = stock.get(color, {}).get(part, 0)
            need_qty = required[color][part] - stock_qty
            to_paint[color][part] = max(need_qty, 0)
    return to_paint

def form_batches(to_paint: Dict[str, Dict[str, int]]) ->
List[Dict]:
    batches = []
    batch_trolley_counts = {
```

```
     'Бампер': 7,
     'Спойлер': 3,
     'Транклид': 3
    }

    batch_capacity = {
        'Бампер передний': 7,
        'Бампер задний': 7,
        'Спойлер': 6,
        'Транклид': 9
    }
    batch_sizes = [13, 13, 13, 13, 13]
    batch_number = 1
    trolley_counts = batch_trolley_counts
    batch_capacity = batch_capacity

    color_batches_needed = {}
    for color, parts in to_paint.items():
        max_batches = 0
        for part, qty in parts.items():
            capacity = batch_capacity.get(part, 1)
            batches_needed_for_part = math.ceil(qty / capacity) if
qty > 0 else 0
            if batches_needed_for_part > max_batches:
                max_batches = batches_needed_for_part
        if max_batches == 0:
           continue
        color_batches_needed[color] = max_batches

    color_queue = [{'color': color, 'remaining': count} for color,
count in color_batches_needed.items()]

    while any(c['remaining'] > 0 for c in color_queue):
        for batch_size in batch_sizes:
            next_color_entry = next((c for c in color_queue if
c['remaining'] > 0), None)
            if next_color_entry is None:
                break
            color = next_color_entry['color']
            parts = to_paint[color]

            batch = {
                'Партия №': batch_number,
                'Цвет': color,
                'Размер партии': batch_size,
                'Бампер передний': min(parts['Бампер передний'] -
(color_batches_needed[color] - next_color_entry['remaining']) *
batch_capacity['Бампер передний'], batch_capacity['Бампер
передний']),
                'Бампер задний': min(parts['Бампер задний'] -
(color_batches_needed[color] - next_color_entry['remaining']) *
batch_capacity['Бампер задний'], batch_capacity['Бампер задний']),
```

```
                'Спойлер': min(parts['Спойлер'] -
(color_batches_needed[color] - next_color_entry['remaining']) *
batch_capacity['Спойлер'], batch_capacity['Спойлер']),
                'Транклид': min(parts['Транклид'] -
(color_batches_needed[color] - next_color_entry['remaining']) *
batch_capacity['Транклид'], batch_capacity['Транклид']),
                'Тип тележек': trolley_counts
            }

            for key in ['Бампер передний', 'Бампер задний',
'Спойлер', 'Транклид']:
                if batch[key] < 0:
                    batch[key] = 0

            batches.append(batch)
            batch_number += 1
            next_color_entry['remaining'] -= 1

            if all(c['remaining'] <= 0 for c in color_queue):
                break

    return batches

required = count_required_parts(queue_df)
stock = count_stock_parts(stocks_df)
to_paint = calc_to_paint(required, stock)
batches = form_batches(to_paint)

print("\nСформированные партии покраски на смену: \n")
for batch in batches:
    print(f"Партия {batch['Партия №']} — Цвет: {batch['Цвет']}:\n"
          f"Передних бамперов: {batch['Бампер передний']}, "
          f"Задних бамперов: {batch['Бампер задний']}, "
          f"Спойлеров: {batch['Спойлер']}, "
          f"Транклидов: {batch['Транклид']}, \n"
          f"Тележек (Бамперы/Спойлеры/Транклиды): {batch['Тип
тележек']} \n")

def get_color_priorities(queue_df):
    color_positions = {}
    for idx, row in queue_df.iterrows():
        color = row['Цвет']
        if color not in color_positions or idx <
color_positions[color]:
            color_positions[color] = idx
    return color_positions

def evaluate_sequence_with_priority(sequence, batches, queue_df,
stock, to_paint):
    color_priorities = get_color_priorities(queue_df)
    priority_penalty = 0
    for i in range(len(sequence)):
        for j in range(i + 1, len(sequence)):
```

```
            color_i = batches[sequence[i]]['Цвет']
            color_j = batches[sequence[j]]['Цвет']
            if color_priorities[color_i] >
color_priorities[color_j]:
                priority_penalty += 10000
    schedule = []
    base_time = 0
    interval =  batches[0]['Размер партии'] * time_btw_telezhka
    for pos, idx in enumerate(sequence):
        batch = batches[idx]
        if pos == 0:
            start_time = base_time
            end_time = time_per_batch
        else:
            start_time = time_per_batch + (pos - 1) * interval
            end_time = start_time + batch['Размер партии'] *
time_btw_telezhka

        schedule.append({
            'Партия №': batch['Партия №'],
            'Цвет': batch['Цвет'],
            'Начало': start_time,
            'Окончание': end_time,
            'Размер партии': batch['Размер партии'],
            # Важно: передавать количество деталей по типам для
корректной проверки
            'Бампер передний': batch.get('Бампер передний', 0),
            'Бампер задний': batch.get('Бампер задний', 0),
            'Спойлер': batch.get('Спойлер', 0),
            'Транклид': batch.get('Транклид', 0),
        })

    delay_penalty = validate_schedule(schedule, queue_df, stock,
to_paint)

    color_switch_penalty = 0
    for i in range(1, len(sequence)):
        if batches[sequence[i]]['Цвет'] != batches[sequence[i -
1]]['Цвет']:
            color_switch_penalty += setup_telezhka_count *
time_btw_telezhka * 1000

    total_penalty = priority_penalty + delay_penalty +
color_switch_penalty
    return total_penalty

def validate_schedule(schedule: List[Dict], queue_df:
pd.DataFrame, stock: Dict[str, Dict[str, int]], to_paint:
Dict[str, Dict[str, int]]) -> float:
    penalty = 0
    start_assembly = assembly_start_delay
    interval = assembly_interval
    color_batches = defaultdict(list)
```

```
    for entry in schedule:
        color_batches.setdefault(entry['Цвет'], []).append(entry)
    for color in color_batches:
        color_batches[color].sort(key=lambda x: x['Окончание'])

    for i, row in queue_df.iterrows():
        color = row['Цвет']
        required_time = start_assembly + i * interval
        stock_by_part = stock.get(color, {'Бампер передний':0,
'Бампер задний':0, 'Спойлер':0, 'Транклид':0}).copy()

        for batch in color_batches.get(color, []):
            if batch['Окончание'] <= required_time:
                for part in ['Бампер передний', 'Бампер задний',
'Спойлер', 'Транклид']:
                    stock_by_part[part] += batch.get(part, 0)
            else:
                break
        needed = {part: i+1 for part in ['Бампер передний',
'Бампер задний', 'Спойлер', 'Транклид']}
        for part, qty_needed in needed.items():
            shortage = qty_needed - stock_by_part.get(part, 0)
            if shortage > 0:
                penalty += shortage * 10000
    return penalty

def eval_individual(individual, batches, queue_df, stock,
to_paint):
    return evaluate_sequence_with_priority(individual, batches,
queue_df, stock, to_paint),

def setup_genetic_algorithm(batches: List[Dict], queue_df:
pd.DataFrame, stock, to_paint):
    creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
    creator.create("Individual", list, fitness=creator.FitnessMin)
    toolbox = base.Toolbox()
    def init_heuristic():
        color_priorities = get_color_priorities(queue_df)
        color_to_batches = defaultdict(list)
        for idx, batch in enumerate(batches):
            color_to_batches[batch['Цвет']].append(idx)
        sorted_colors = sorted(color_to_batches.keys(), key=lambda
c: color_priorities.get(c, 10000))
        sorted_indices = []
        for color in sorted_colors:
sorted_indices.extend(color_to_batches[color])
        return sorted_indices

    def group_mutation(individual):
      color_groups = defaultdict(list)
      for idx in individual:
        color = batches[idx]['Цвет']
        color_groups[color].append(idx)
```

```
        colors = list(color_groups.keys())
        random.shuffle(colors)
        new_individual = []
        for color in colors:
            new_individual.extend(color_groups[color])
        return new_individual,

    toolbox.register("init_heuristic", init_heuristic)
    toolbox.register("individual", tools.initIterate,
creator.Individual, toolbox.init_heuristic)
    toolbox.register("population", tools.initRepeat, list,
toolbox.individual)
    toolbox.register("group_mutate", group_mutation)
    toolbox.register("evaluate",
functools.partial(eval_individual, batches=batches,
queue_df=queue_df, stock=stock, to_paint=to_paint))
    toolbox.register("mate", tools.cxOrdered)
    toolbox.register("mutate", tools.mutShuffleIndexes, indpb=0.2)
# Вероятность мутации
    toolbox.register("select", tools.selTournament, tournsize=5)
    return toolbox

def lns_local_search(sequence, batches, queue_df, stock, to_paint,
max_iter, destroy_size):
    def evaluate(seq):
        return evaluate_sequence_with_priority(seq, batches,
queue_df, stock, to_paint,)

    best_seq = sequence[:]
    best_score = evaluate(best_seq)

    for iteration in range(max_iter):
        start = random.randint(0, len(sequence) - destroy_size)
        end = start + destroy_size
        destroyed = best_seq[start:end]
        remainder = best_seq[:start] + best_seq[end:]
        random.shuffle(destroyed)
        new_seq = remainder[:start] + destroyed +
remainder[start:]
        new_score = evaluate(new_seq)
        if new_score < best_score:
            best_seq = new_seq
            best_score = new_score
            print(f"LNS улучшил результат на итерации {iteration},
штраф: {best_score}")
    return best_seq

def optimize_batch_order(batches: List[Dict], queue_df:
pd.DataFrame,  stock, to_paint,) -> List[int]:
    print("Запуск генетического алгоритма...")
    toolbox = setup_genetic_algorithm(batches, queue_df,  stock,
to_paint,)
    population = toolbox.population(n=100)
```

```
    algorithms.eaMuPlusLambda(population, toolbox, mu=100,
lambda_=100, cxpb=0.8, mutpb=0.05, ngen=200, verbose=True)
    best_ga = tools.selBest(population, 1)[0]
    print("Запуск LNS для улучшения решения...")
    improved_seq = lns_local_search(best_ga, batches, queue_df,
stock, to_paint, max_iter = 300, destroy_size = 5)
    return improved_seq

def print_final_schedule(batches, optimized_order):
    base_time = 0
    interval = batches[0]['Размер партии'] * time_btw_telezhka
    prev_color = None

    print("\nОптимальное расписание партий:")
    for pos, idx in enumerate(optimized_order):
        batch = batches[idx]
        if pos == 0:
            start_time = base_time
            end_time = time_per_batch
            empty_trolleys = 0
        else:
            start_time = time_per_batch + (pos - 1) * interval
            end_time = start_time + batch['Размер партии'] *
time_btw_telezhka
            empty_trolleys = 2 if batch['Цвет'] != prev_color else
0

        print(f"Партия {batch['Партия №']} ({batch['Цвет']}): "
              f"{timedelta(minutes=round(start_time))} -
{timedelta(minutes=round(end_time))} | "
              f"Пустых тележек: {empty_trolleys}")
        prev_color = batch['Цвет']

if __name__ == "__main__":
  optimized_order = optimize_batch_order(batches, queue_df, stock,
to_paint)
  print_final_schedule(batches, optimized_order)

def format_time(minutes):
    # Преобразует минуты во время ЧЧ:ММ
    td = timedelta(minutes=round(minutes))
    hours, remainder = divmod(td.seconds, 3600)
    minutes = remainder // 60
    return f"{hours:02}:{minutes:02}"

def export_schedule_to_excel(batches, optimized_order,
output_file):
    rows = []
    prev_color = None
    base_time = 0
    interval = batches[0]['Размер партии'] * time_btw_telezhka
    for pos, idx in enumerate(optimized_order):
        batch = batches[idx]
```

```
        if pos == 0:
            start_time = base_time
            end_time = time_per_batch
            empty_trolleys = 0
            color_switch = "Нет"
        else:
            start_time = time_per_batch + (pos - 1) * interval
            end_time = start_time + batch['Размер партии'] *
time_btw_telezhka
            empty_trolleys = 2 if batch['Цвет'] != prev_color else
0
            color_switch = "Да" if batch['Цвет'] != prev_color
else "Нет"

        rows.append({
            "Порядок в плане": pos + 1,
            "Цвет": batch['Цвет'],
            "Передних бамперов": batch.get('Бампер передний', 0),
            "Задних бамперов": batch.get('Бампер задний', 0),
            "Спойлеров": batch.get('Спойлер', 0),
            "Транклидов": batch.get('Транклид', 0),
            "Размер партии (тележек)": batch['Размер партии'],
            "Время начала (чч:мм)": format_time(start_time),
            "Время окончания (чч:мм)": format_time(end_time),
            "Пустых тележек": empty_trolleys,
            "Смена цвета": color_switch
        })
        prev_color = batch['Цвет']
    df = pd.DataFrame(rows)
    df.to_excel(output_file, index=False)
    wb = load_workbook(output_file)
    ws = wb.active

    header_fill = PatternFill(start_color="C6E0B4",
end_color="C6E0B4", fill_type="solid")  # нежно-зеленый

    for col_num, column_cells in enumerate(ws.iter_cols(min_row=1,
max_row=1), 1):
        cell = column_cells[0]
        cell.alignment = Alignment(wrap_text=True,
horizontal='center', vertical='center')
        cell.fill = header_fill
    for col in ws.columns:
        max_length = 0
        col_letter = get_column_letter(col[0].column)
        for cell in col:
            try:
                cell_value = str(cell.value)
                if cell_value is None:
                    length = 0
                else:
                    length = len(cell_value)
                if length > max_length:
```

```
                    max_length = length
            except:
                pass
        adjusted_width = (max_length + 2)
        ws.column_dimensions[col_letter].width = adjusted_width
    wb.save(output_file)
    print(f"\nГотовый план сохранён в файл: {output_file}")

output_file = config.get('PATHS', 'output_file',
fallback='D:/VKR/Plan.xlsx')
export_schedule_to_excel(batches, optimized_order, output_file)
```

ПРИЛОЖЕНИЕ Б

Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)

Факультет: «Робототехника и комплексная автоматизация»

Кафедра: «Компьютерные системы автоматизации производства

Выпускная квалификационная работа на тему:

«Система планирования и диспетчеризации линии окраски пластиковых деталей»

Выполнил студент РК9-81Б: **Максимов Д.И.**

Руководитель проекта: **к.т.н. Овсянников М.В.**

Консультант проекта: **Буханов С.А.**

2025 г.

Цель и объект ВКР

Цель: разработка алгоритма оптимизации порядка покраски партий пластиковых деталей с учетом технологических ограничений и требований к производственному процессу, а также создание программного обеспечения для реализации данного алгоритма.

Объект исследования: линия окраски пластиковых деталей автомобильного завода АО МАЗ «Москвич».

2

Задачи ВКР

- Анализ существующей системы планирования и диспетчеризации линии окраски пластиковых деталей;
- Анализ современных методов планирования и диспетчеризации производственных процессов;
- Анализ основных алгоритмов и методов теории расписания, применяющихся для планирования производств;
- Разработка алгоритмического решения задачи оптимизации планирования на основе методов теории расписаний, способного учитывать сложные производственные ограничения и приоритеты;
- Реализация программного кода для автоматизации процесса планирования расписания покраски;
- Тестирование разработанной системы на реальных данных завода «Москвич» и оценка эффективности предложенного подхода.

3

Целевое обследование

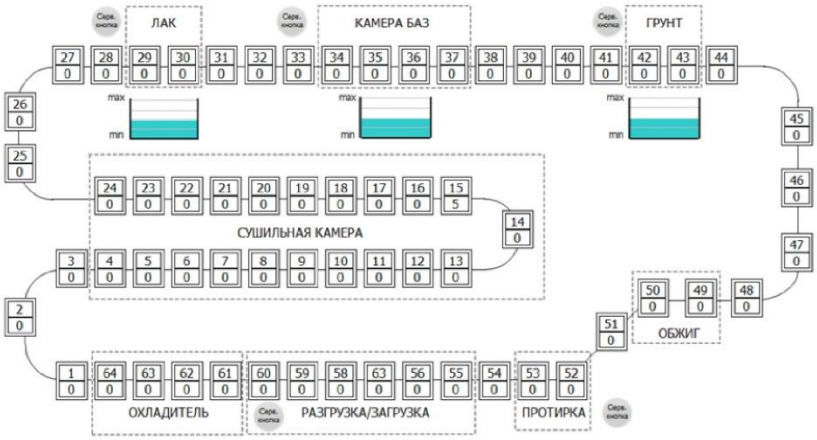


Схема линии окраски пластиковых деталей АО МАЗ “Москвич”

4

Целевое обследование линии окраски

5

Анализ существующей системы планирования

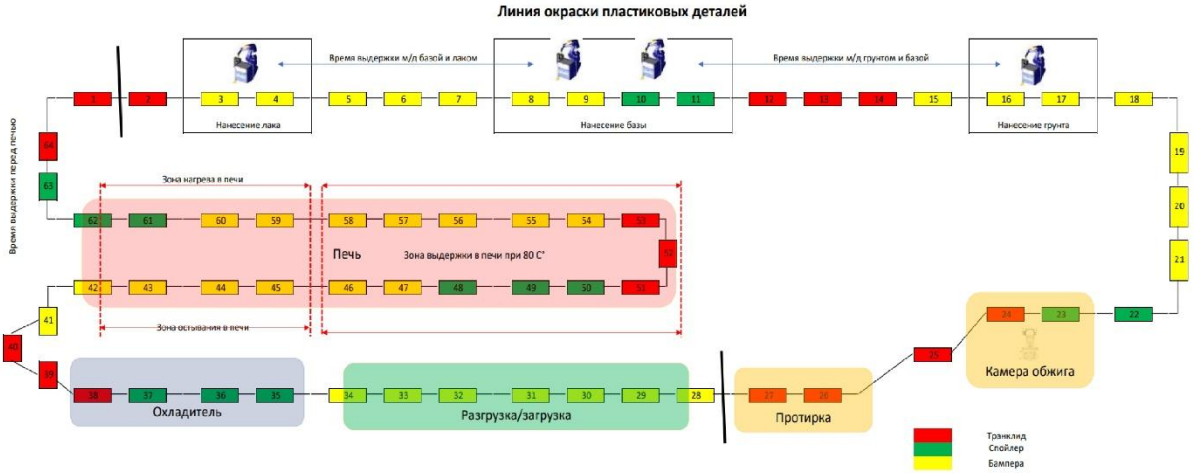


Схема назначения тележек на линии окраски

6

Анализ существующей системы планирования

Требуемое количество телег с учетом двойного прохода транклид	95 авто в смену	126 авто в смену	138 авто в смену
Комплект бамперов на 1 авто (2 бампера)	95	126	138
Спойлер	47,5	63	69
Транклид	63,3	84	92
Итого	205,8	273,0	299,0
Процент перекраса	4%	4%	4%
Итого с учетом перекраса	214,0667	283,92	310,96
Требуемая производительность телег/час	33,8	44,9	49,1
Фактическая производительность телег/час	36,5	36,5	36,5
Коэффициент использования оборудования	93%	123%	135%
КИО без двойного прохода транклид	78%	104%	114%

Требуемые производственные мощности

7

Существующие решения в области планирования производств

1) Классическое MRP/ERP-планирование
2) Модели математического программирования
3) Системы диспетчерского управления реального времени (RTDCS)
4) Системы оперативного управления и диспетчеризации

8

Методы и алгоритмы теории расписаний

Классические методы планирования:

1) Целочисленное линейное программирование (ILP)

2) Эвристика и метаэвристика:

- Генетические алгоритмы
- Поиск с большими окрестностями (LNS)

Методы для адаптивного и оперативного планирования:

1) Стохастическое программирование с несколькими сценариями (SSP)
2) Стохастическое динамическое программирование (SDP)
3) Планирование с подвижным горизонтом (RHP)

9

Выбор языка программирования

Преимущества программного кода:

1) Фокус на алгоритмической составляющей
2) Интеграция с Excel
3) Гибкость и скорость разработки

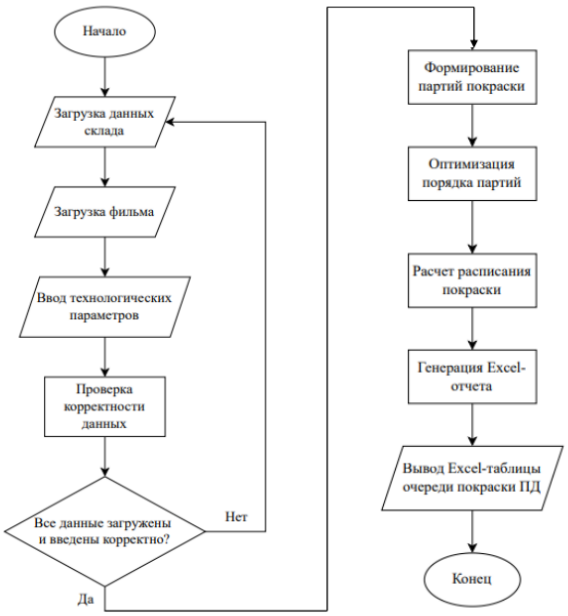
Преимущества Python:

1) Богатая экосистемы библиотек
2) Простота и скорость разработки:
3) Академическая популярность

10

Алгоритм работы программы

1) Загрузка и проверка данных
2) Формирования списка партий покраски
3) Оптимизация порядка партий
4) Расчет расписания
5) Генерация выходного файла
6) Завершение работы программы



11

Тестирование работы программы

ФИЛЬМ DKD +

Дата входа на сборку M131	Номер по порядку	VIN	Тара с кузовом	Модель	Комплектация	Тип	Цвет	Жижа	Мотор	Номер мотора	тип	TCM +
17.12.2024	1	X7JA15BAC1020816	CKD	M3	Flagship	CVT	Голубой + Черн	СУХОЙ			CKD	#Н/Д
17.12.2024	2	X7JA13BAC1019148	CKD	M3	Luxury heated	CVT	Красный	СУХОЙ			CKD	#Н/Д
17.12.2024	3	X7JA13BAC1020302	CKD	M3	Luxury heated	CVT	Красный	СУХОЙ			CKD	#Н/Д
17.12.2024	4	X7JA13BAC1015353	CKD	M3	Luxury heated	CVT	Красный	СУХОЙ			CKD	#Н/Д
17.12.2024	5	X7JA13BAM1019311	CKD	M3	Luxury heated	MT	Черный	СУХОЙ			CKD	#Н/Д
17.12.2024	6	X7JA13BAC1020254	CKD	M3	Luxury heated	CVT	Красный	СУХОЙ			CKD	#Н/Д
17.12.2024	7	X7JA13BAM1019316	CKD	M3	Luxury heated	MT	Черный	СУХОЙ			CKD	#Н/Д
17.12.2024	8	X7JA13BAC1020182	CKD	M3	Luxury heated	CVT	Красный	СУХОЙ			CKD	#Н/Д
17.12.2024	9	X7JA13BAM1019329	CKD	M3	Luxury heated	MT	Черный	СУХОЙ			CKD	#Н/Д
17.12.2024	10	X7JA13BAC1020341	CKD	M3	Luxury heated	CVT	Красный	СУХОЙ			CKD	#Н/Д
17.12.2024	11	X7JA13BAC1020205	CKD	M3	Luxury heated	CVT	Красный	СУХОЙ			CKD	#Н/Д
17.12.2024	12	X7JA13BAC1020344	CKD	M3	Luxury heated	CVT	Красный	СУХОЙ			CKD	#Н/Д
17.12.2024	13	X7JA15BAC1020635	CKD	M3	Flagship	CVT	Красный + Черн	СУХОЙ			CKD	#Н/Д
17.12.2024	14	X7JA13BAC1019826	CKD	M3	Luxury heated	CVT	Красный	СУХОЙ			CKD	#Н/Д
17.12.2024	15	X7JA13BAC1020211	CKD	M3	Luxury heated	CVT	Красный	СУХОЙ			CKD	#Н/Д
17.12.2024	16	X7JA13BAM1018708	CKD	M3	Luxury heated	MT	Серый	СУХОЙ			CKD	#Н/Д
17.12.2024	17	X7JA13BAC1020277	CKD	M3	Luxury heated	CVT	Красный	СУХОЙ			CKD	#Н/Д
17.12.2024	18	X7JA13BAM1016107	CKD	M3	Luxury heated	MT	Серый	СУХОЙ			CKD	#Н/Д
17.12.2024	19	X7JA13BAC1020273	CKD	M3	Luxury heated	CVT	Красный	СУХОЙ			CKD	#Н/Д
17.12.2024	20	X7JA13BAM1016226	CKD	M3	Luxury heated	MT	Серый	СУХОЙ			CKD	#Н/Д
17.12.2024	21	X7JA13BAC1020328	CKD	M3	Luxury heated	CVT	Красный	СУХОЙ			CKD	#Н/Д
17.12.2024	22	X7JA13BAM1016162	CKD	M3	Luxury heated	MT	Серый	СУХОЙ			CKD	#Н/Д
17.12.2024	23	X7JA13BAC1015371	CKD	M3	Luxury heated	CVT	Красный	СУХОЙ			CKD	#Н/Д
17.12.2024	24	X7JA13BAM1019307	CKD	M3	Luxury heated	MT	Черный	СУХОЙ			CKD	#Н/Д
17.12.2024	25	X7JA13BAC1020872	CKD	M3	Luxury heated	CVT	Красный	СУХОЙ			CKD	#Н/Д

Пример фильма сборки кузовов для тестирования

12

Тестирование работы программы

Номенклатура	Цвет	Кол-во на складе
КОРПУС ПЕРЕДНЕГО БАМПЕРА Белый	WH1-0	3
КОРПУС ПЕРЕДНЕГО БАМПЕРА Голубой	LB1-0	3
КОРПУС ПЕРЕДНЕГО БАМПЕРА Красный	RE1-0	3
КОРПУС ПЕРЕДНЕГО БАМПЕРА Серый	GR1-0	3
КОРПУС ПЕРЕДНЕГО БАМПЕРА Черный	BK1-0	3
КОРПУС ЗАДНЕГО БАМПЕРА Белый	WH1-0	3
КОРПУС ЗАДНЕГО БАМПЕРА Голубой	LB1-0	3
КОРПУС ЗАДНЕГО БАМПЕРА Красный	RE1-0	3
КОРПУС ЗАДНЕГО БАМПЕРА Серый	GR1-0	3
КОРПУС ЗАДНЕГО БАМПЕРА Черный	BK1-0	3
СПОЙЛЕР Белый	WH1-0	6
СПОЙЛЕР Голубой	LB1-0	6
СПОЙЛЕР Красный	RE1-0	6
СПОЙЛЕР Серый	GR1-0	6
СПОЙЛЕР Черный	BK1-0	6
ТРАНКЛИД Белый	WH1-0	6
ТРАНКЛИД Голубой	LB1-0	6
ТРАНКЛИД Красный	RE1-0	6
ТРАНКЛИД Серый	GR1-0	6
ТРАНКЛИД Черный	BK1-0	6

Запасы покрашенных деталей на складе

13

Тестирование работы программы

Порядок в плане	Цвет	Передних бамперов	Задних бамперов	Спойлеров	Транклид	Размер партии (тележек)	Время начала (чч:мм)	Время окончания (чч:мм)	Пустых тележек	Смена цвета
1	Голубой	3	3	0	0	13	00:00	01:41	0	Нет
2	Красный	7	7	6	9	13	01:41	02:02	2	Да
3	Красный	7	7	6	5	13	02:02	02:22	0	Нет
4	Черный	7	7	6	9	13	02:22	02:43	2	Да
5	Красный	7	7	6	9	13	02:43	03:03	2	Да
6	Красный	7	7	6	9	13	03:03	03:24	0	Нет
7	Красный	4	4	6	0	13	03:24	03:44	0	Нет
8	Красный	0	0	2	0	13	03:44	04:05	0	Нет
9	Красный	7	7	6	9	13	04:05	04:25	0	Нет
10	Красный	7	7	6	0	13	04:25	04:46	0	Нет
11	Серый	7	7	6	9	13	04:46	05:06	2	Да
12	Серый	7	7	6	9	13	05:06	05:27	0	Нет
13	Красный	7	7	6	9	13	05:27	05:47	2	Да
14	Серый	7	7	6	9	13	05:47	06:08	2	Да
15	Черный	6	6	4	1	13	06:08	06:29	2	Да
16	Серый	7	7	6	4	13	06:29	06:49	2	Да
17	Серый	7	7	6	0	13	06:49	07:10	0	Нет
18	Серый	1	1	4	0	13	07:10	07:30	0	Нет
19	Серый	7	7	6	9	13	07:30	07:51	0	Нет

Результат тестирования программы

14

Заключение

- Проведен анализ принципа работы, планирования и диспетчеризации линии окраски пластиковых деталей АО МАЗ «Москвич»;
- Проведен анализ современных методов планирования и диспетчеризации производственных процессов;
- Проведен анализ основных алгоритмов и методов теории расписания, которые применяются для планирования производств;
- Разработан алгоритм планирования с учетом приоритетов и производственных ограничений, сочетающий в себе генетический алгоритм и методы локального поиска;
- Реализован программный код, позволяющий автоматизировать процесс планирования и составления расписания на смену;
- Проведено тестирование разработанной системы на реальных производственных данных завода АО МАЗ «Москвич».

15